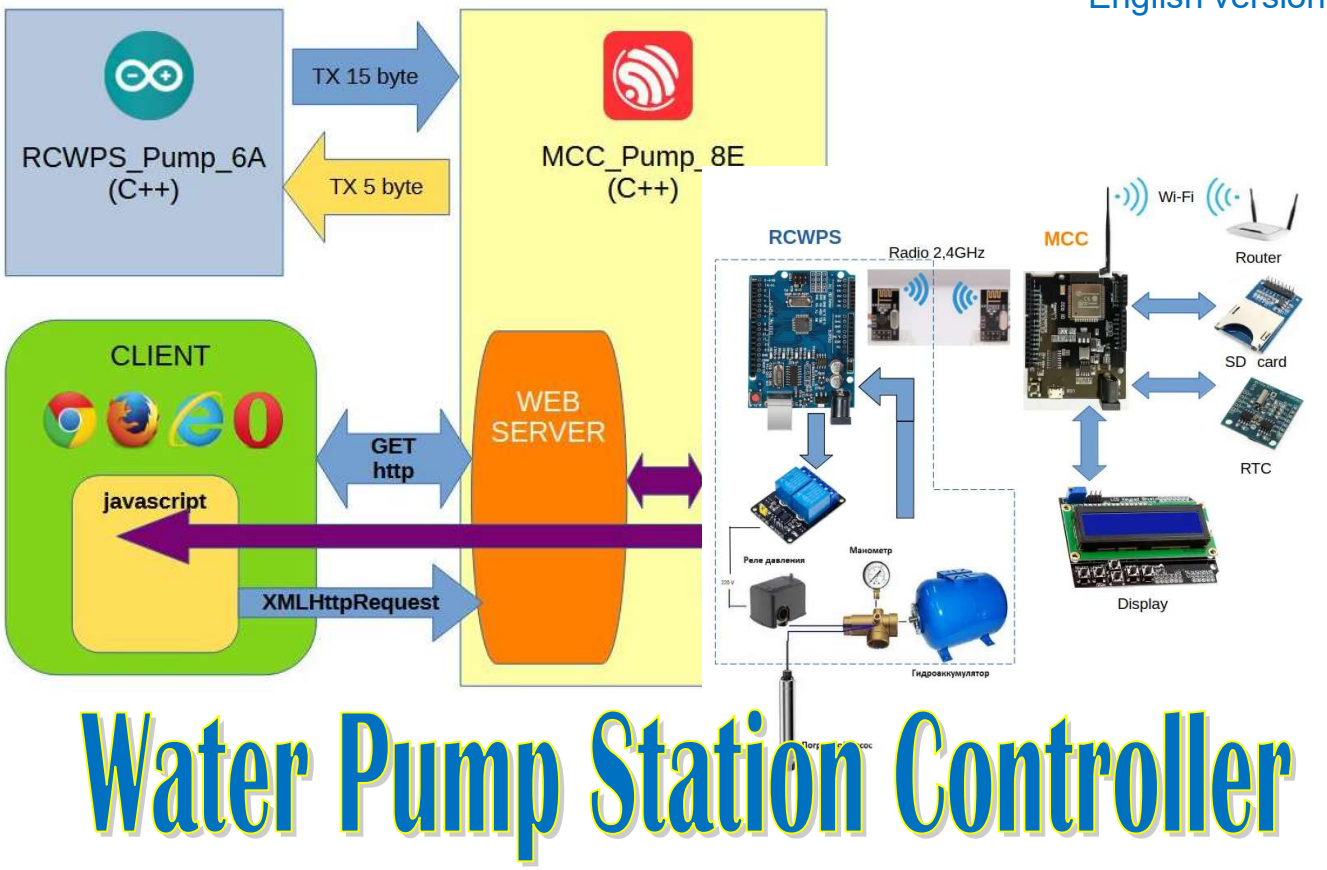
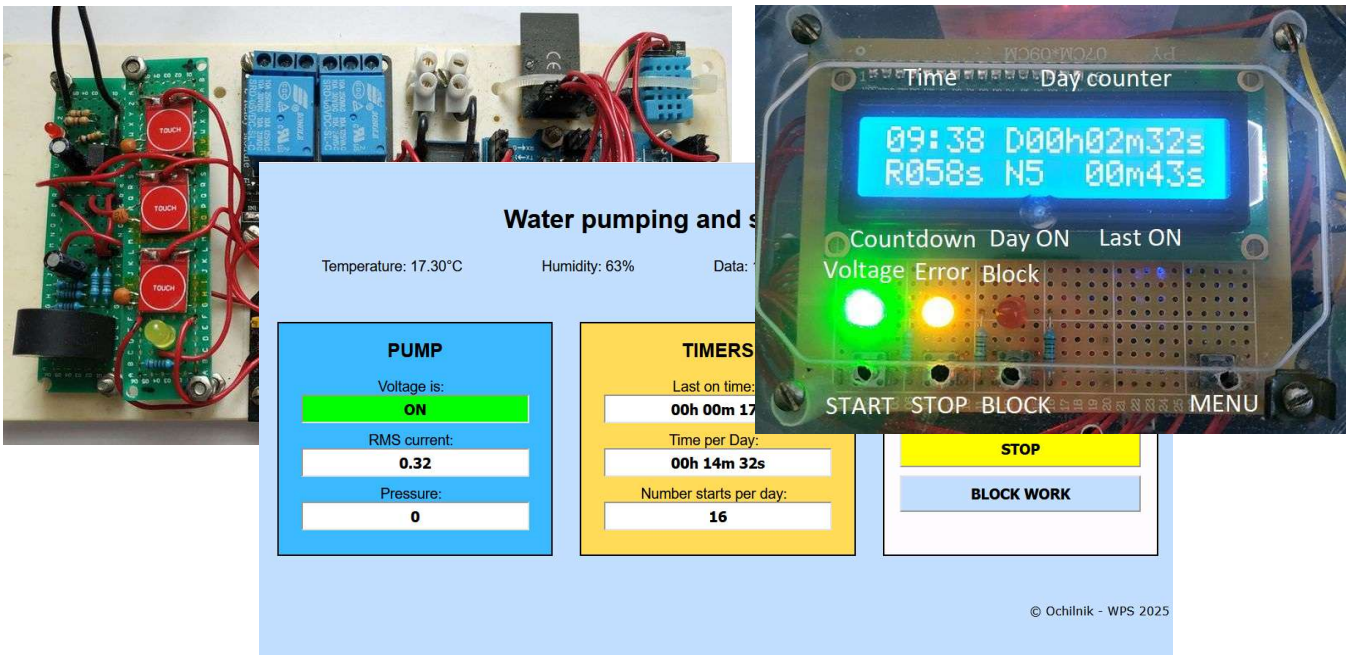
English version



Water Pump Station Controller



Description of the project of the controller of the home system of remote wireless monitoring of the state and control of the Pumping and Storage Station of water with a remote control and a web server

ochilnik 2025 v1.2

Water Pumping Station (WPS) Control System.

Contents

1 Introduction 4
1.1 Scope 4
1.2 Limitations of Standard Systems 4
2 Requirements Analysis 5
2.1 Functional Requirements 5
2.2 Hardware Requirements 5
2.3 Software Requirements 5
3 WPS Control System Architecture 6
4 Hardware Implementation 8
4.1 WPS Equipment and Modifications 8
4.2 The RCWPS Controller 8
4.2.1 Controller Board 8
4.2.2 Relay Module 9
4.2.3 Transceiver Module 9
4.2.4 Temperature and Humidity Sensor 10
4.2.5 AC Voltage Detection Module 10
4.2.6 AC Current Measurement Module 11
4.2.7 Power Supply Module 12
4.2.8 Final Assembly 12
4.3 The MCC Controller 13
4.3.1 Controller Board 13
4.3.2 Transceiver Module 14
4.3.3 SD Card Module 14
4.3.4 User Interface Module 15
4.3.5 RTC Module 16
4.3.6 Logic Level Converter 16
4.3.7 Power Supply 16
4.3.8 Final Assembly 16
5 Software 18
5.1 Software Architecture 18
5.2 Packet-Based Communication 19
5.3 RCWPS Controller Firmware 21
5.3.1 Included Libraries 21
5.3.2 Variables and Constants 21
5.3.3 Data Packet Handling 22

5.3.4 Manual Run Timer 24
5.3.5 Control 25
5.3.6 Fault Handling 26
5.4 MCC Controller 26
5.4.1 Included Libraries 26
5.4.2 Переменные, константы и объекты 27
5.4.3 Packet Reception / Transmission 28
5.4.4 Display 30
5.4.5 Display Backlight Timer 33
5.4.6 Real-Time Clock 34
5.4.7 Operating Time Counters 35
5.4.8 Control and Indication 36
5.4.9 Fault Handling 37
5.4.10 SD-MMC Memory 38
5.4.11 Wi-Fi 38
5.4.12 Web Server 39
5.4.13 Web Client 43
6 Operational Description 48
7 Future Upgrades 52

1 Introduction

1.1 Scope

In many households across Ukraine, a consistent and reliable supply of drinking water remains a significant challenge. Consequently, homeowners are often required to implement their own solutions. A common approach is the installation of a Water Pumping Station (WPS).

The configuration of these systems can vary. Brief descriptions of common setups include:

- Well-Based System: A pump delivers water from a well directly into the home's plumbing. This setup typically includes a mechanical water filter, a pressure switch, a pressure tank, and the necessary shut-off valves and fittings.
- Storage Tank System: Water from a main municipal supply line fills a large storage tank. The WPS then draws water from this tank and delivers it to the home's plumbing. The additional equipment is generally the same as in the well-based system.
- Hybrid Systems: Various configurations that are a development or combination of the methods described above.

1.2 Limitations of Standard Systems

While the equipment described above solves the basic challenge of water supply, it also introduces several failure modes that can lead to hazardous conditions, including:

- Downstream Leaks: A water leak occurring after the pump. As these systems are often installed in remote locations (e.g., basements or outbuildings), a leak may go undetected for a significant amount of time.
- Dry Running: A lack of water at the source (well or tank). In this scenario, the pump continues to operate without water ("run dry"), causing it to overheat and fail. Standard systems are not typically equipped with dry-run protection sensors.
- Loss of Air Pre-charge: A drop in the pressure tank's air pressure. This causes the pump to run for longer cycles, sometimes continuously. The pressure switch's cut-off point is calibrated to the sum of the tank's air pre-charge and the desired upper water pressure. As air loss from the tank is a common occurrence, this leads to frequent pump cycling and premature wear.
- Ambient Temperature Fluctuation: Changes in the surrounding temperature can also alter the response time of the pressure switch's setpoints.
- Diaphragm Rupture: A punctured diaphragm (or bladder) in the pressure tank leads to a situation similar to the loss of air pre-charge.
- Pressure Switch Blockage: Clogging of the hydraulic inlet port of the pressure switch, leading to incorrect readings.
- Pressure Switch Contact Failure: Sparking (arcing), overheating, and subsequent contact welding in the pressure switch that controls the pump. This can cause the pump to run continuously, leading to relay overheating, fire hazards, uncontrolled system over-pressurization, and resulting pipe or fitting ruptures.

The analysis of these common failure modes highlighted the clear need to develop and manufacture a dedicated control system for Water Pumping Stations (WPS).

2 Requirements Analysis

2.1 Functional Requirements

The controller must address the following functional requirements:

- Monitor the 220V AC power supply to the pump motor.
- Monitor the current draw of the pump motor.
- Measure the water or air pressure within the pressure tank.
- Track the pump's runtime per operating cycle.
- Track the total cumulative pump runtime over a 24-hour period.
- Monitor the ambient temperature.
- Detect water leaks.
- Provide manual start/stop control for the pump.
- Provide a manual lockout function to prevent pump activation.
- Generate alarms and warnings for system fault conditions.
- Provide a local display for key system parameters.
- Provide access to system parameters via a built-in web server.

2.2 Hardware Requirements

To meet the specified requirements, the ideal solution is a cost-effective, general-purpose controller supported by a wide selection of off-the-shelf expansion modules.

The selected controller must not only fulfill the functional requirements outlined above but also address the following technical hardware challenges:

- Conversion of external signals to levels compatible with the controller's inputs (e.g., 220V AC to 5V DC).
- Conversion of the variable AC current signal into a proportional RMS (Root Mean Square) value, scaled to the controller's analog input range.
- An executive device (e.g., a relay or contactor) to switch the pump motor circuit.
- Capabilities for remote data transmission and reception.
- An integrated RTC for accurate timekeeping and scheduling.
- An interface for connecting to a local TCP/IP network.
- Sufficient non-volatile memory to store the web server application code.
- Logic level shifting to ensure signal compatibility between the main controller and its various modules.
- A power supply unit (PSU) with enough capacity to power the main controller and all connected modules.

2.3 Software Requirements

Given the unique nature of this project, the controller's operating software must be custom-developed. The development process requires a suitable Integrated Development Environment (IDE) and access to pre-existing software libraries for the connected hardware modules.

3 WPS Control System Architecture

The design of the system architecture is primarily determined by the physical environment of a typical installation, specifically the separation between the equipment and the user.

In a typical setup, the Water Pumping Station (WPS) equipment is located in a protected, remote area such as a basement. This location has 220V AC power, but it often lacks network connectivity (e.g., Wi-Fi).

Conversely, the ideal location for monitoring the system and interacting with it is in the main living area of the house. This location has both a power source and reliable Wi-Fi network access.

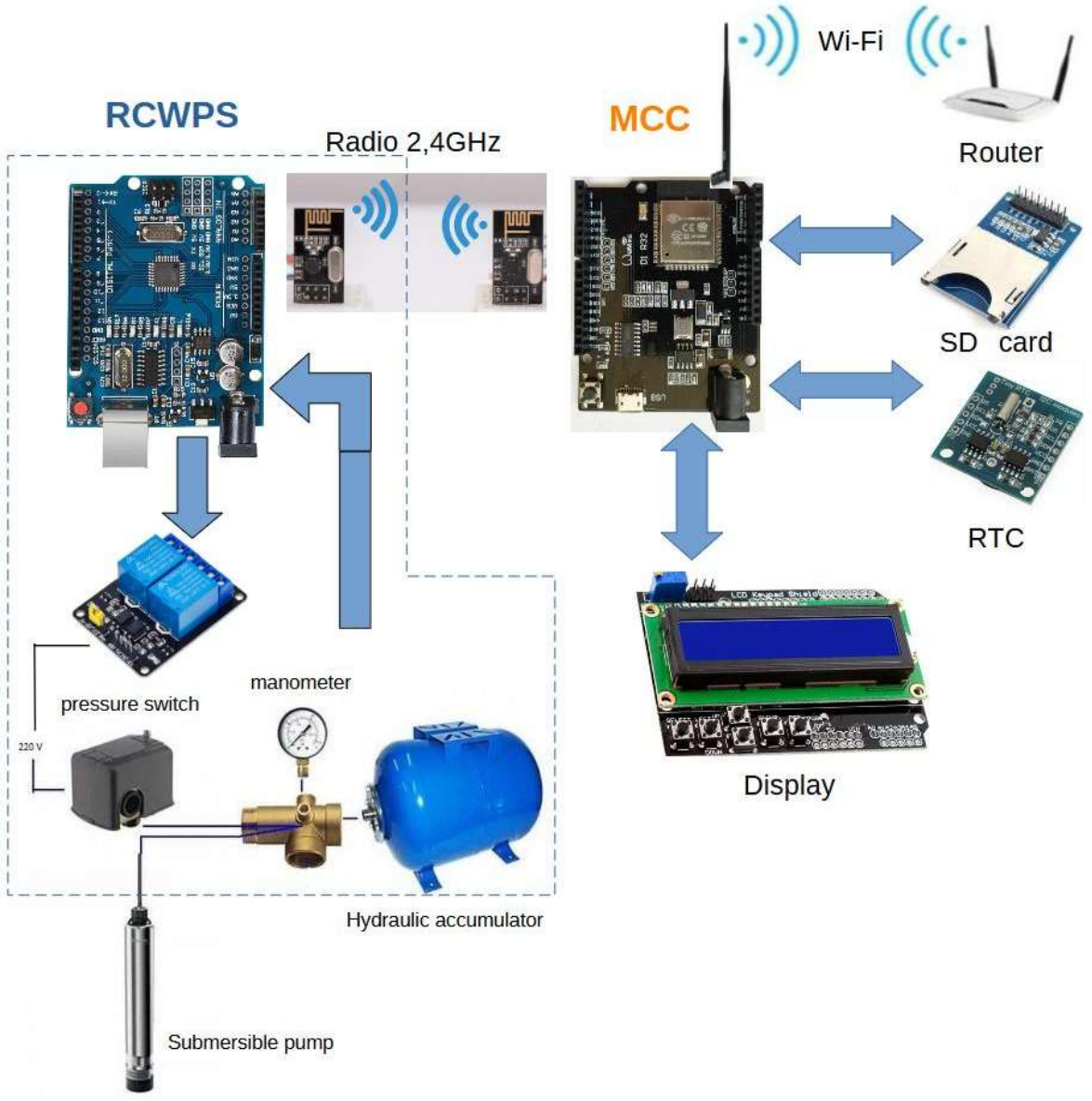


Figure 1 — WPS Control System Block Diagram

Based on these environmental constraints, the WPS Control System is composed of two main controller units. The first unit, located with the WPS equipment, is the Remote Controller Water Pump Station (RCWPS). The second unit, situated in the main

living area, is the Master Central Controller (MCC). The system's functions and their corresponding hardware modules are distributed between these two units.

As shown in the system block diagram above (Figure 1), data from the WPS equipment is fed into the RCWPS, which is responsible for the following tasks:

- Performing initial signal processing.
- Monitoring for fault and alarm conditions.
- Handling local control commands.
- Controlling the interposing relays in the pump motor circuit.
- Packaging data for transmission.
- Communicating with the MCC (transmitting and receiving data).

Communication between the two controllers is established over a 2.4 GHz wireless link.

The MCC controller is responsible for the following high-level tasks:

- Communicating with the RCWPS (receiving data and sending commands).
- Parsing and processing the incoming data packets.
- Managing runtime counters and timers.
- Interfacing with the Real-Time Clock (RTC) for accurate timekeeping.
- Managing data storage and retrieval from non-volatile memory.
- Executing advanced fault detection logic based on the collected data.
- Processing user input from the local interface (e.g., buttons).
- Controlling all audible and visual alarms.
- Driving the local display to show system status and parameters.
- Transmitting diagnostic data to the serial port.
- Handling read/write operations for the SD card (e.g., for data logging).
- Hosting the built-in web server to provide the remote user interface.

Additionally, the MCC controller manages the connection to the local Wi-Fi network.

4 Hardware Implementation

4.1 WPS Equipment and Modifications

The equipment of a standard Water Pumping Station (WPS) can be categorized into two groups: hydraulic components (pump, filters, piping, pressure switch) and electrical components (electric motor, circuit breaker, pressure switch, and associated wiring). Note that the pressure switch belongs to both categories as it is both a hydraulic sensor and an electrical switch.

This project introduces only minor modifications to the pump motor's power circuit, primarily for the connection of monitoring circuits.

As a best practice, even if the full control system is not installed, an interposing contactor (K1) should be added to the existing circuit between the pressure switch and the motor. This contactor offloads the high-current motor load from the pressure switch's contacts, thereby preventing common failures such as arcing, burning, and contact welding.

The installation of contactor K1 also enables remote control of the WPS. This is achieved by installing additional control relays: KV1 and KV2 for remote Start/Stop functionality, and relay KV3 for a remote pump lockout function.

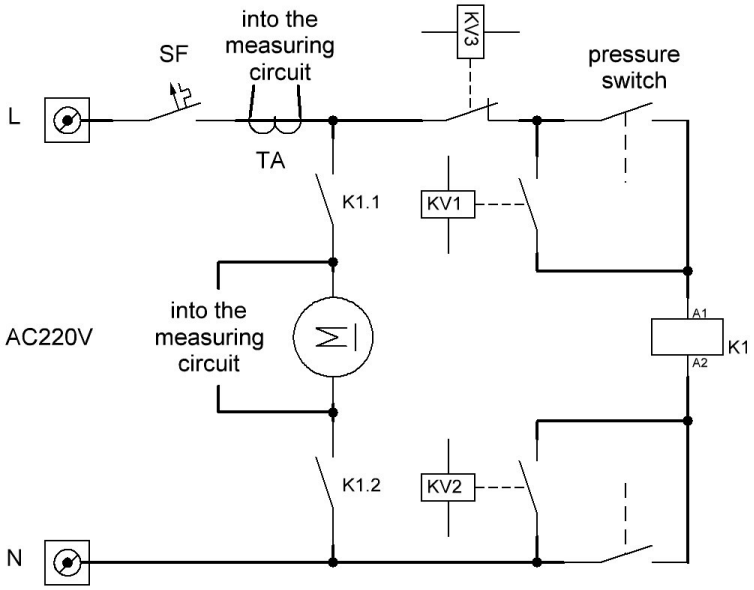


Figure 2 — Electrical Connection Diagram for the WPS

4.2 The RCWPS Controller

4.2.1 Controller Board

An Arduino UNO clone based on the ATmega328 microcontroller was selected as the remote controller. This board was chosen for its low cost and for meeting all essential requirements, including:

- A cost-effective, mass-produced controller with a wide selection of compatible, off-the-shelf expansion modules.

- A free and well-supported Integrated Development Environment (Arduino IDE) with extensive, readily available software libraries.
- Clock Speed: 16 MHz
- Memory: 32 KB Flash, 2 KB SRAM, 1 KB EEPROM
- I/O: 14 digital input/output pins
- Analog Inputs: 6 analog input channels
- An onboard USB-B port with a built-in USB-to-UART converter for programming.
- A standard DC barrel jack accepting a 7-12V power supply.

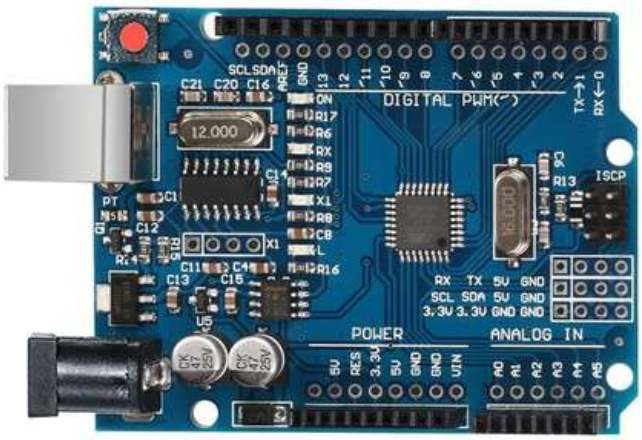


Figure 3 – Arduino UNO Controller Board

4.2.2 Relay Module

A 2-channel, 5V optically-isolated relay module is used to interface the controller's low-voltage signals with the 220V AC power circuit.

This module consists of two onboard relays, each rated to switch a load of up to 10A at 220V AC. The relay coils operate on a 5V DC supply from the controller.

The overall system design requires three control relays (KV1, KV2, and KV3). This 2-channel module provides the relays for the Remote Start (KV1) and Lockout (KV3) functions. A third, separate relay was added to the circuit to implement the Remote Stop (KV2) function.

Figure 4 – 2-Channel 5V Optically-Isolated Relay Module

4.2.3 Transceiver Module

The system uses the **NRF24L01+**, a 2.4 GHz wireless transceiver module. It is based on a single-chip RF transceiver designed for the license-free 2.4 GHz ISM

(Industrial, Scientific, and Medical) band. The chip integrates a frequency synthesizer, power amplifier (PA), crystal oscillator, modulator, demodulator, and a hardware protocol engine known as Enhanced ShockBurst™.

This module autonomously handles the entire data packet transmission and reception process, including automatic acknowledgment and retransmission, which significantly offloads the main microcontroller.

Key Specifications:

- Frequency Range: 2.4 GHz to 2.5 GHz
- Channels: 125
- Maximum Data Rate: 2 Mbps
- Output Power: 0 dBm (for the standard NRF24L01+ version)
- Supply Voltage: 1.9V to 3.6V

Figure 5 – The NRF24L01+ 2.4GHz Wireless Transceiver Module

4.2.4 Temperature and Humidity Sensor

Figure 6 – DHT11 Temperature and Humidity Sensor

4.2.5 AC Voltage Detection Module

The following sections describe the custom-built modules for this project.

To determine if the pump motor is running, the system monitors the presence of the 220V AC supply at its terminals. To achieve this, a dedicated 220V AC interface module was designed and hand-soldered.

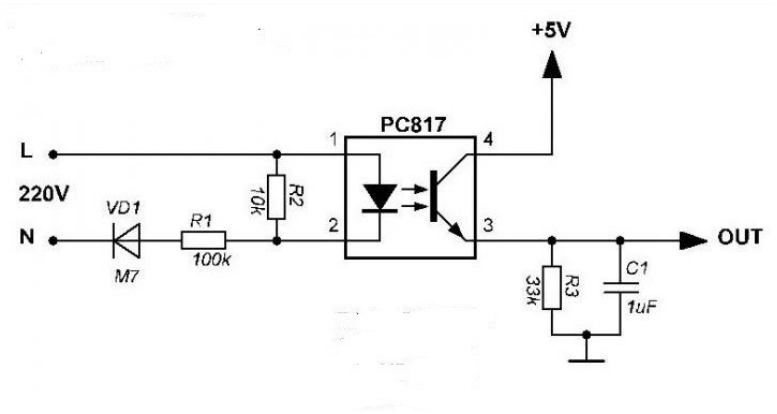


Figure 7 – 220V AC to Controller Interface Schematic

4.2.6 AC Current Measurement Module

The same principle of custom design applies to the module used for measuring the pump motor's AC current.

A conventional hardware approach for this task would require a sophisticated circuit with operational amplifiers to perform true RMS (Root Mean Square) conversion. However, to simplify the hardware for this cost-effective application, the complex signal integration is instead handled by the software.

As a result, the hardware circuit is greatly simplified. Its only function is to safely scale the AC signal from the current sensor into the controller's analog input range and apply a DC bias to establish a virtual ground.

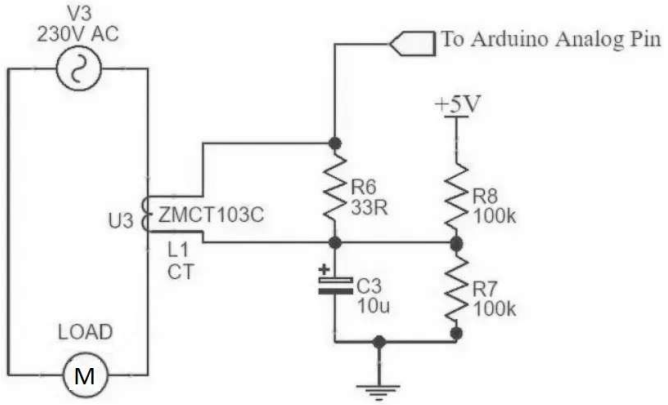


Figure 8 – Current Transformer to Controller Interface Schematic

Both of the interface circuits described above (for AC voltage and current detection) are combined onto a single Printed Circuit Board (PCB), which serves as the bottom board.

A second PCB is mounted on top of it, creating a user interface board. This top board features capacitive touch buttons for pump control (START, STOP, and LOCKOUT) and status LEDs to indicate system warnings and alarms.

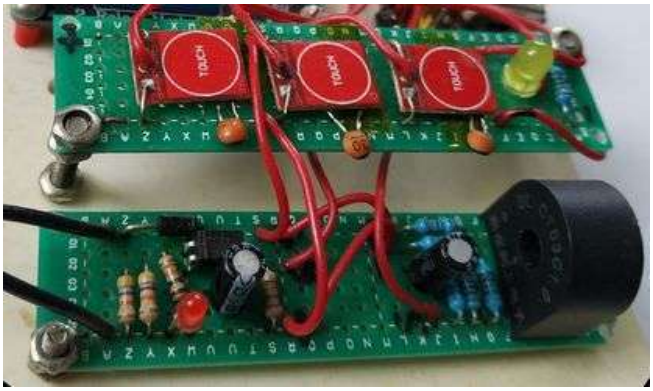


Figure 9 – The User Interface Board and AC Interface Board

4.2.7 Power Supply Module

Power for the controller and its associated modules is provided by an HW-131 breadboard power supply module.

This module's key specifications are:

- Input Voltage: 6.5V to 12V (via a standard DC5521 barrel jack)
- Output Voltages: 5V and 3.3V
- Maximum Output Current: 700 mA (This is the total combined current available across both voltage rails).

Figure 10 – HW-131 Power Supply Module

4.2.8 Final Assembly

The complete RCWPS controller unit is formed by installing the main controller board and all associated modules in a common hermetic box.

Design Note: It is important to note that functional reliability and cost effectiveness were the primary design goals for this prototype.

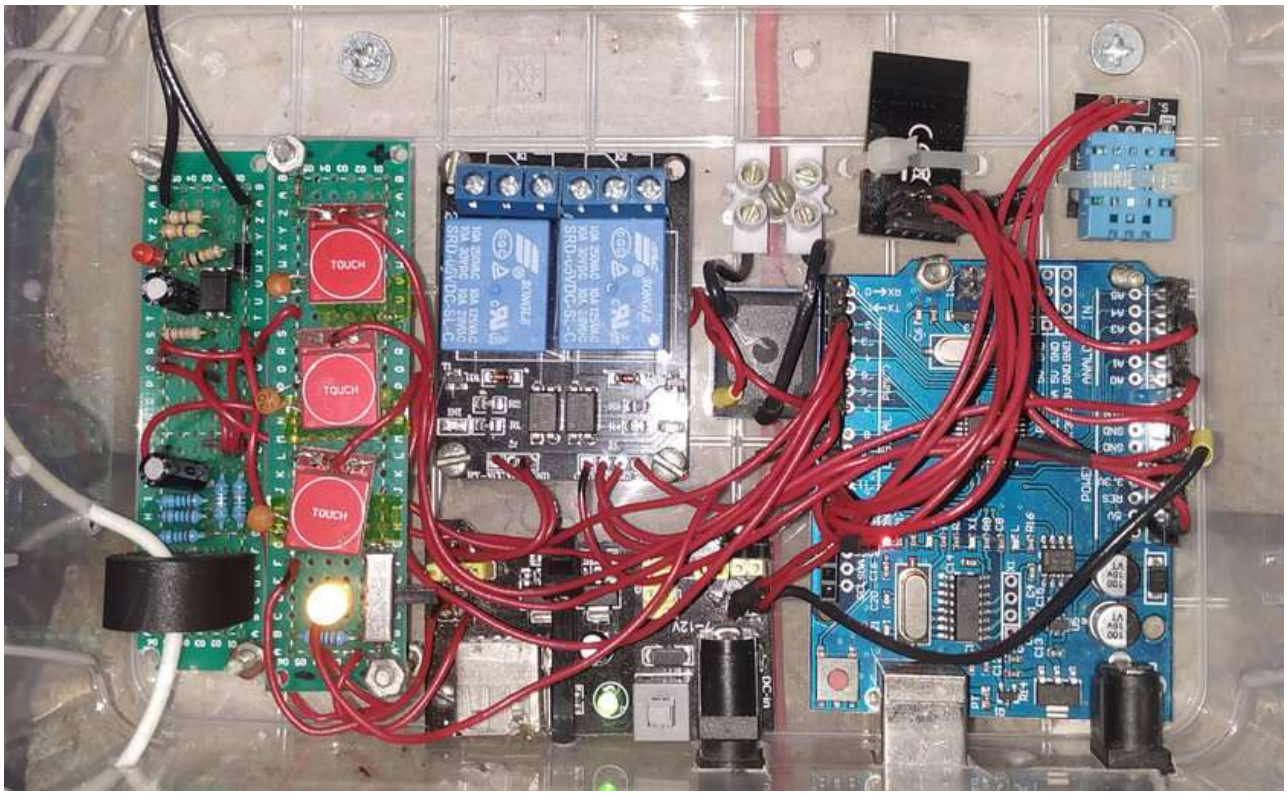


Figure 11 – The Assembled RCWPS Controller Unit

4.3 The MCC Controller

4.3.1 Controller Board

The Master Central Controller (MCC) is responsible for significantly more demanding computational and communication tasks. For this purpose, a WEMOS D1 R32 controller board, based on the Espressif ESP32 microcontroller, was selected. Its key specifications are:

- A mass-produced, cost-effective controller;
- Compatibility with standard Arduino modules;
- Programmable using the Arduino IDE;
- Availability of extensive software libraries;
- Clock speed of 240 MHz;
- Memory: 4 MB Flash, 512 KB SRAM;
- 20 digital input/output pins;
- 6 analog inputs;
- Communication interfaces: SPI, I2C, I2S, IR, UART, PWM;
- Wi-Fi 802.11 b/g/n and Bluetooth 4.2 LE;
- Onboard Wi-Fi antenna;
- A micro-USB port with a built-in USB-to-UART converter for programming;
- 3.3V logic level for the controller and I/O pins;
- 7-12V power input via a standard DC barrel jack.

Figure 12 – Wemos D1 R32 Controller Board

4.3.2 Transceiver Module

An identical 2.4GHz NRF24L01+ wireless module is used to establish communication with the remote controller (RCWPS).

Figure 13 – NRF24L01+ 2.4GHz Wireless Module

4.3.3 SD Card Module

The selected controller features a built-in SD-MMC interface, which frees up general-purpose I/O (GPIO) pins for other peripherals. Since the controller operates at a 3.3V logic level, the SD card can be connected directly without requiring any logic level shifting. For the physical interface, a standard SD to micro SD adapter was modified by soldering wires directly to its contacts. A micro SD card is then inserted into this adapter.

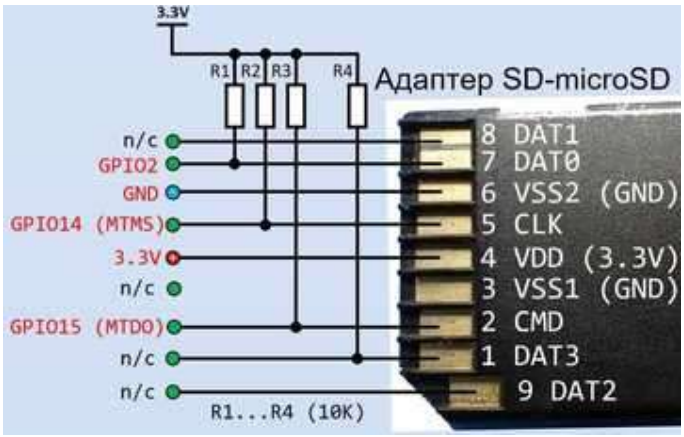


Figure 14 – SD-MMC Interface Schematic

4.3.4 User Interface Module

The user interface board was assembled from a 16x2 character LCD, an I2C-to-LCD converter module, tactile buttons, and status LEDs. The converter module translates the display's native parallel interface into a two-wire I2C serial interface. The buttons (Start, Stop, Lockout) have the same function as their counterparts on the remote controller.

START: Initiates a timed manual run of the pump for one minute. This is a safety feature that bypasses the pressure switch.

STOP: Manually stops the pump. This command has no effect if the pump was started automatically by the pressure switch.

LOCKOUT: Prevents the pump from starting. If the pump is already running, this command also stops it. This function acts as a toggle; the button must be pressed again to disable the lockout.

Since the system operates automatically and these buttons are intended for manual override in special cases, they are recessed within the enclosure and require a small tool or pusher to be actuated.

MENU: (Unique to the MCC) This button cycles through the various information screens on the LCD. It also activates the display's backlight for a three-minute period. For ease of access, it is implemented as a capacitive touch button, using the bottom-right mounting screw of the case as the touch electrode.

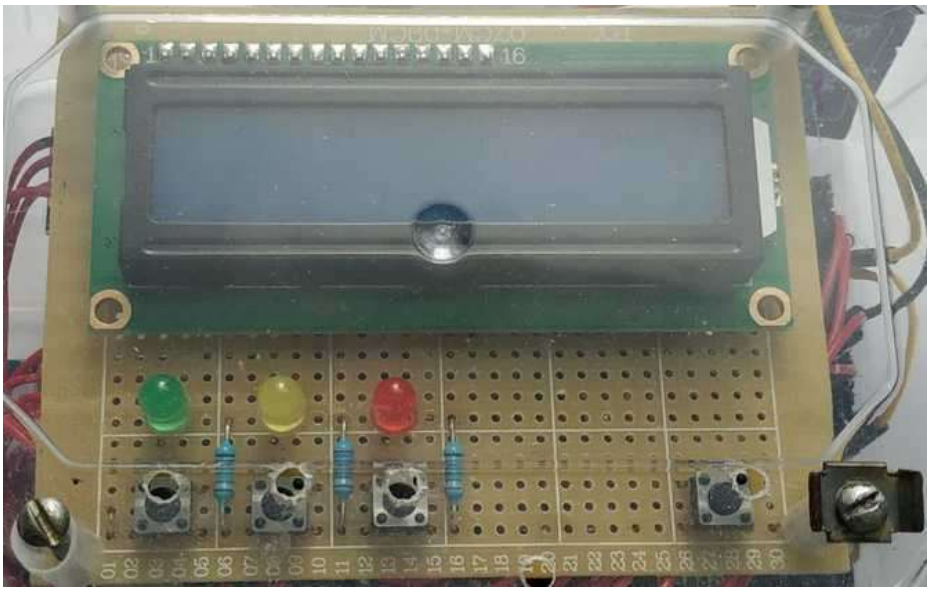


Figure 15 – The Display and Control Board

To accommodate more information than the display can show at once, the data is organized into the following distinct screens, grouped by function:

Screen 1: Counters and Timers

Screen 2: Measured Parameters

Screen 3: Error Messages

4.3.5 RTC Module

The system uses a DS1307 I2C Real-Time Clock module, which integrates both the DS1307 RTC IC and an AT24C32 32 Kbit EEPROM chip on a single board.

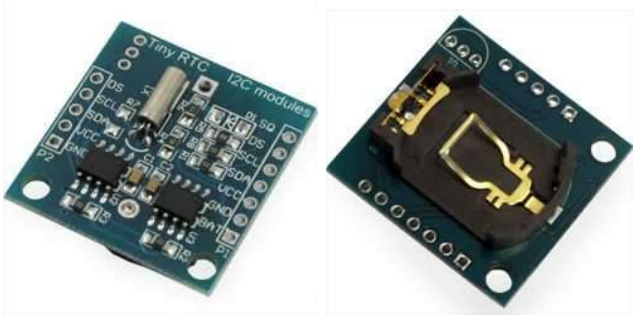


Figure 16 – DS1307 RTC Module

Both the display converter module and the RTC module connect to the controller via the I2C serial bus, which uses only two wires (SDA and SCL).

4.3.6 Logic Level Converter

Both the display converter and RTC modules require a 5V power supply to operate correctly. Since the main controller operates at a 3.3V logic level, a bidirectional logic level converter is required to ensure safe and reliable communication between the 5V peripherals and the 3.3V controller.

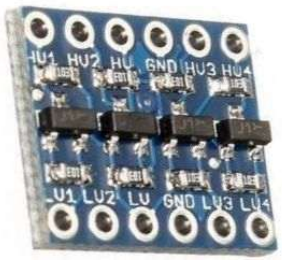


Figure 17 – 4-Channel Bidirectional Logic Level Converter

4.3.7 Power Supply

Power for the auxiliary boards and modules is provided by two separate voltage regulator boards, one for 5V and one for 3.3V, both based on the 1117-series linear regulators. The main controller board is powered by its own separate, onboard voltage regulator.

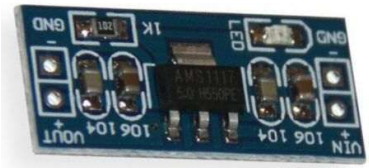


Figure 18 – 1117-Series Voltage Regulator Module

4.3.8 Final Assembly

The complete MCC controller unit was created by assembling all the internal boards and modules into a single enclosure.

As previously stated, it is important to reiterate that this is a functional prototype. The primary considerations during its construction were reliability and cost-effectiveness, using readily available components. Therefore, external appearance was not a design priority.

Figure 19 – The Assembled MCC Controller

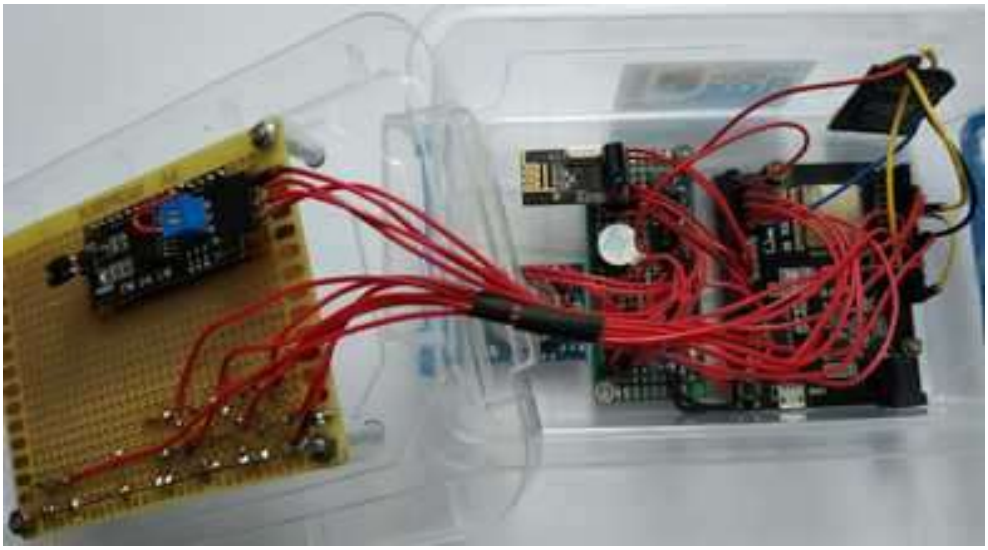


Figure 20 – MCC Controller with its Cover Open

5 Software

5.1 Software Architecture

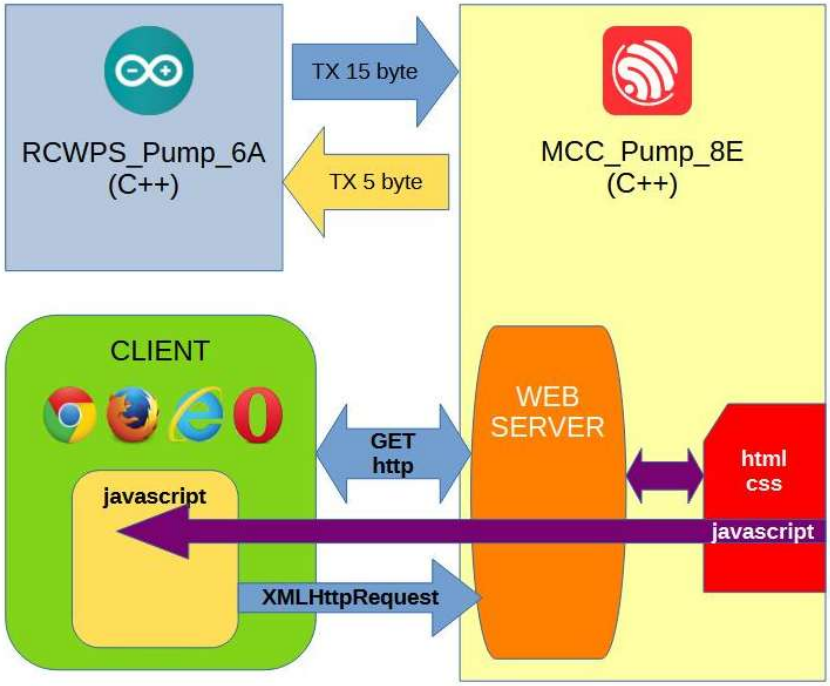


Figure 21 – System Data Flow Diagram

This section outlines the fundamental principles of data exchange within the system. The diagram above illustrates the primary entities involved in the data flow: the RCWPS controller, the MCC controller, the integrated web server, and the end-user client.

As previously mentioned, communication between the two controllers occurs via packet-based transmissions over a wireless radio link. The structure and purpose of these data packets will be detailed in a later section.

Based on this data, the controllers manage the WPS equipment, trigger system alarms, and update the local display with current parameters.

In addition to its primary control loop, the MCC controller runs an embedded web server connected to the local network via its built-in network interface. When a client connects to the server's IP address on port 80, the server responds by sending an HTML/CSS page and a client-side script, both of which are read from the SD card. The client's web browser renders this page, and the downloaded script then fetches updated system parameters every second using Asynchronous JavaScript and XML (AJAX) requests. Commands from the client are sent to the server using the HTTP GET method.

In summary, the WPS control system consists of four primary software components:

- The embedded firmware for the RCWPS controller, written in C++.
- The embedded firmware for the MCC controller, also written in C++.
- The server-side application (the web server itself).
- The client-side web application, consisting of HTML, CSS, and JavaScript.

5.2 Packet-Based Communication

Data is exchanged between the controllers in packets transmitted over the wireless link. The data packet sent from the RCWPS to the MCC is 15 bytes long, while the packet sent from the MCC to the RCWPS is 5 bytes long.

The variable names used in the following tables are taken directly from the source code. For a detailed description of each variable's purpose, please refer to the comments within the corresponding program listings.

Table 1 - MCC to RCWPS Data Packet Structure

BYTE	Bit / Name Variables							
	Bit 0	Bit 1	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7
0	alarmMCC	commandOnPump	commandOffPump	commandBlock	-	-	-	-
1	Reserved							
2	Reserved							
3	Reserved							
4	Reserved							

Table 2 – Error Code Definitions for the codErrMCC Variable

bit	Error Description codErrMCC
0	one switching time is longer than 10 minutes
1	working time per day longer than 60 minutes
2	the consumed current is above the nominal value
3	the consumed current is below the nominal value
4	the temperature is above 30C
5	Reserved
6	Reserved
7	Reserved

Table 3 - RCWPS to MCC Data Packet Structure

BYTE	Bit / Name Variables							
	Bit 0	Bit 1	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7
0	voltagePumpPin	buttonOnPumpPin	buttonOffPumpPin	buttonBlockPin	-	-	-	-
1	alarmRCWPS	onPumpPin	blockPumpPin	-	-	-	-	-
2	high byte (currentRMS * 100)							
3	low byte (currentRMS * 100)							
4	Reserved							
5	Reserved							
6	high byte (Temperature * 10)							
7	low byte (Temperature * 10)							
8	byte Humidity							
9	Error code RCWPS (codErrRCWPS[8])							
10	Rest time to OFF Pump - cycle							
11	Reserved							
12	Reserved							
13	Reserved							
14	Reserved							

Table 4 – Error Code Definitions for the codErrRCWPS Variable

bit	Error Description codErrRCWPS
0	If output onPump is ON (inverse) and no voltage
1	If output blockPump is ON (inverse) and voltage on
2	Reserved
3	Reserved
4	Reserved
5	Reserved
6	Reserved
7	Reserved

5.3 RCWPS Controller Firmware

The firmware for the RCWPS controller was developed in C++ using the Arduino Integrated Development Environment (IDE).

https://github.com/Ochilnik/WPS/blob/master/RCWPS_Pump_6A.ino.

The software for the controllers is built on a cyclical execution model. The Arduino C++ environment provides a special function, loop(), specifically for this purpose. All code placed within this function is executed repeatedly in a continuous cycle. It is critical that the algorithm within the loop is non-blocking and its execution time remains consistent, as long delays would disrupt the timing of the main program cycle.

Prior to the execution of the main loop(), a setup() function is called once after the controller is powered on or reset.

This structure naturally divides the program's functional code into two main parts: one-time initialization and the main repetitive logic.

5.3.1 Included Libraries

- <SPI.h>: The library for communicating over the SPI bus, which is used by the nRF24L01+ module.
- <nRF24L01.h>, <RF24.h>: Libraries for operating the nRF24L01+ wireless transceiver module.
- <Adafruit_Sensor.h>: The base library required for Adafruit's unified sensor driver framework.
- <DHT.h>, <DHT_U.h>: Libraries for interfacing with the DHT11 temperature and humidity sensor.
- <EmonLib.h>: The "EmonLib" library, used for non-invasive AC current measurement.

5.3.2 Variables and Constants

```
#define DHTPIN 17                    // Digital pin connected to the DHT sensor
#define DHTTYPE    DHT11             // DHT 11

RF24 radio(9, 10);                   // Pins D9, D10 for CE, CSN on Arduino UNO
DHT_Unified dht(DHTPIN, DHTTYPE);
String bits = "";
EnergyMonitor emon1;                 // Create an instance

const uint64_t addr_tx = 0xAABBCCDD11LL;  // Transmit pipe address
const uint64_t addr_rx = 0xAABBCCDD22LL;  // Receive pipe address
const uint8_t num_ch = 119;          // Channel number

bool commandAlarm;                   // command from MCC - Alarm
bool commandOnPump;                  // command from MCC - Pump ON
bool commandOffPump;                 // command from MCC - Pump OFF
bool commandBlock;                   // command from MCC - Block Pump
bool stateAlarm, alarmMCC, alarmRCWPS;   // Current States signals from RCWPS
bool codErrRCWPS[8] = {0};           // Array of errors codes from RCWPS

uint8_t data_tx[15] = {0};           // Array data to transmit
uint8_t data_rx[5] = {0};            // Array data to receive
uint8_t data_tx_previos[15] = {0};       // Array data that was transmitted last time
uint8_t cycleTimer = 14;             // Number of timer cycles for the desired interval: 60s / 4.19424s = 14.3
uint8_t cycle = cycleTimer;          // Current number of timer cycles
//uint8_t codErrMCC;                 // code of errors from MCC

float currentCalibration = 27.0;
```

```
float currentRMS;

unsigned long measurement_timestamp = 0;
unsigned long interval = 4000ul;  // DHT measurement interval in milliseconds

//***** PINOUT *****
uint8_t voltagePumpPin = 2; // Voltage is supplied to the motor pump
uint8_t buttonOnPumpPin = 4; // button pump on is pressed
uint8_t buttonOffPumpPin = 5; // button pump off is pressed
uint8_t buttonBlockPin = 3; // triger blocking pump is on
uint8_t blockPumpPin = 6; // command blocking pump
uint8_t onPumpPin = 7; // command pump on
uint8_t alarmPin = 8; // signal attention
uint8_t currentPumpPin = 14; // current is consumed motor pump

void setup(){
// ***** Settings INPUT/OUTPUT
  pinMode(voltagePumpPin, INPUT); // Voltage is supplied to the pump motor
  pinMode(buttonOnPumpPin, INPUT); // button pump on is pressed
  pinMode(buttonOffPumpPin, INPUT); // button pump off is pressed
  pinMode(buttonBlockPin, INPUT); // triger blocking pump is on
  pinMode(blockPumpPin, OUTPUT); // command pump off
  digitalWrite(blockPumpPin, !LOW);
  pinMode(onPumpPin, OUTPUT); // command pump on
  digitalWrite(onPumpPin, !LOW);
  pinMode(alarmPin, OUTPUT); // signal attention
}
```

Listing 1 – Variables and Constants

5.3.3 Data Packet Handling

This function represents the core communication logic of the controller. As the listing demonstrates, a significant block of pre-configuration code precedes the main operational loop.

The commented-out parameters have been intentionally left in the code to facilitate easier debugging and tuning.

```
void setup(){
// --- Radio Interface Configuration ---
  Serial.println("Receiver/Transmitter ON");
  radio.begin();                        // Initialize the radio module
  delay(2000);
  //radio.setPayloadSize(7);              // Set the static payload size in bytes (1-32)
  radio.setDataRate(RF24_250KBPS);         // Set the data rate: RF24_1MBPS, RF24_2MBPS, RF24_250KBPS
  switch(radio.getDataRate()){             // Verify the currently set data rate
    case RF24_1MBPS   : Serial.println("setDataRate  1 MBPS"); break;
    case RF24_2MBPS   : Serial.println("setDataRate  2 MBPS"); break;
    case RF24_250KBPS : Serial.println("setDataRate  250 KBPS"); break;
  }
  radio.setCRCLength(RF24_CRC_8);          // Set the CRC length to 8 or 16 bits
  radio.setChannel(num_ch);                // Set the radio channel
  radio.setAutoAck(true);                  // Enable auto-acknowledgment
  radio.setPALevel(RF24_PA_MAX);           // Set Power Amplifier level: RF24_PA_MIN, RF24_PA_LOW, RF24_PA_HIGH,
RF24_PA_MAX

  // --- Receiver Configuration ---
  //radio.enableDynamicPayloads();         // Enable dynamic payload size on the receiver
  radio.openReadingPipe(1, addr_rx);       // Open a reading pipe on the defined address
  //closeReadingPipe();                    // Closes a reading pipe that was previously opened
  //radio.enableAckPayload();              // Enable sending of custom acknowledgment payloads
  radio.startListening();                  // Set the module to receiver mode

  // --- Transmitter Configuration ---
  radio.setRetries(5, 15);                 // Set the retry delay and number of retries (delay = (5+1)*250us, 15 retries)
  radio.openWritingPipe(addr_tx);          // Open a writing pipe on the defined address
  //radio.stopListening();                 // Sets the module to transmitter mode
```

```
  //radio.powerUp();                      // Enter full power mode
  //radio.powerDown();                    // Enter low power consumption mode
}
void loop(){
// --- Receiver Logic ---
  if(radio.available()){                  // If data is available in the RX buffer...
    radio.read(&data_rx, sizeof(data_rx));  // Read the data from the buffer into the data_rx array
    //RX BYTE 0
    alarmMCC = (data_rx[0] >> 0) & 0x01;     // Bit 0. Remote Command to Alarm
    commandOnPump = (data_rx[0] >> 1) & 0x01; // Bit 1. Remote Command to On Pump
    commandOffPump = (data_rx[0] >> 2) & 0x01; // Bit 2. Remote Command to Off Pump
    commandBlock = (data_rx[0] >> 3) & 0x01; // Bit 3. Remote Command to Block work Pump
    //RX BYTE 1
    //
  }

// --- Transmitter Logic ---
  // Prepare data
  //TX BYTE 0
  data_tx[0] =  (digitalRead(buttonBlockPin) << 3) | (digitalRead(buttonOffPumpPin) << 2) |
                (digitalRead(buttonOnPumpPin) << 1) | digitalRead(voltagePumpPin);

  //TX BYTE 1
  data_tx[1] =  (!digitalRead(blockPumpPin) << 2) |
                (!digitalRead(onPumpPin) << 1) | alarmRCWPS;

  //TX BYTES 2-3: Motor Current
  currentRMS = emon1.calcIrms(1480);         // Calculate Irms only
  int packI = (int)round(currentRMS * 100);
  // Pack the current value into two bytes
  data_tx[2] = (packI >> 8) & 0xFF;          // High byte
  data_tx[3] = packI & 0xFF;                 // Low byte

  //TX BYTES 4-5: Reserved for Water Pressure (Analog Input)

  //TX BYTES 6-8: Temperature and Humidity
  if (millis() - measurement_timestamp >= interval) {
    sensors_event_t event;
    // Get temperature event and print its value.
    dht.temperature().getEvent(&event);
    if (!isnan(event.temperature)) {
      int packTemp = (int)round(event.temperature * 10);
      // Pack the temperature into two bytes
      data_tx[6] = (packTemp >> 8) & 0xFF;   // High byte
      data_tx[7] = packTemp & 0xFF;          // Low byte
    }
    // Get humidity event and print its value.
    dht.humidity().getEvent(&event);
    if (!isnan(event.relative_humidity)) {
      data_tx[8] = (int)round(event.relative_humidity);
    }
    measurement_timestamp = millis();        // Update the timestamp for the next reading
  }

  //TX BYTE 9: Error code RCWPS
  data_tx[9] = 0;
  for (int i = 0; i < 8; i++) {
    if (codErrRCWPS[i]) {
      data_tx[9] |= (1 << i);                // Set the corresponding bit
    }
  }

  //TX BYTE 10: Rest time to OFF Pump
  data_tx[10] = cycle;

  // Only transmit if the data has changed
  if(!dataAreEqual(data_tx, data_tx_previos, sizeof(data_tx))) {
    radio.stopListening();                   // Switch to transmitter mode
    radio.write(&data_tx, sizeof(data_tx));  // Transmit the data packet
    toSerialTX();
    radio.startListening();                  // Switch back to receiver mode
  }
```

```
  // Update the 'previous data' buffer for the next comparison
  for (uint8_t i = 0; i < sizeof(data_tx); i++) {
    data_tx_previos[i] = data_tx[i];
  }
}
```

Listing 2 - Data Packet Handling

In the main program loop, the first step is to receive and parse the incoming data packet, populating the corresponding global variables. Next, the outgoing data packet is prepared for transmission. The nRF24L01+ library function can transmit up to 32 bytes per packet (this project uses 15 bytes).

To use the available packet space efficiently, multiple boolean variables are packed into single bytes. Conversely, integer variables are split into high and low bytes for transmission.

Although a new data packet is prepared during each program cycle, it is only transmitted if its contents differ from the previously sent packet. To achieve this, after the comparison and potential transmission, the newly prepared packet is copied into a temporary buffer to be used for the next cycle's comparison.

This approach significantly reduces radio frequency congestion and lowers the controller's power consumption without compromising communication reliability.

5.3.4 Manual Run Timer

Time-based operations in the controller must be handled without blocking the main program loop. Therefore, to manage the timed manual pump run, the microcontroller's hardware Timer1 is used in conjunction with standard timer control libraries.

The timer is configured to generate an overflow interrupt at its maximum interval, calculated as follows:

- Clock Prescaler: 1024. (16 MHz System Clock / 1024 = 15,625 Hz)
- Timer Tick Period: 1 / 15,625 Hz = 0.000064 seconds (64 µs)
- Time to Overflow: 65,536 ticks * 64 µs/tick = 4.1943 seconds

Even at this maximum setting, the interval is insufficient for the desired one-minute runtime. Therefore, a software counter (cycleTimer = 14) is used to count the number of timer overflows, allowing the system to achieve the target interval of approximately one minute.

A key principle of using hardware interrupts is that the Interrupt Service Routine (ISR) must execute as quickly as possible. Therefore, the ISR in this code only sets a flag or decrements the counter; the main program loop is responsible for handling the resulting logic.

The timer is enabled and disabled within the dedicated functions that handle the manual pump start and stop commands.

```
void setup(){
// ***** Settings Timer1
  noInterrupts();                 // disable all interrupts
  TCCR1A = 0;                       // timer/counter control registers
  TCCR1B = 0;
  TCNT1 = 0;                       // register for preloading a value (set to 0)
  TIMSK1 |= (1 << TOIE1);              // enable timer overflow interrupt ISR
  interrupts();                   // enable all interrupts
```

```
}

void startTimer1(){
  TCCR1B |= (1 << CS10)|(1 << CS12);    // Start timer 1 with prescaler 1024
}

void stopTimer1(){
  TCCR1B &= ~(1 << CS10 | 1 << CS11 | 1 << CS12);    // Stop Timer1
}

ISR(TIMER1_OVF_vect) {  // interrupt service routine for counter overflow
  if(cycle>0){
     cycle--;
  } else {
     stopPump();
  }
```

Listing 3 - Timer Module

5.3.5 Control

Now, the control part of the program only needs to detect a forced command from the buttons or the central controller and call the corresponding function.

```
void startPump(){
  digitalWrite(onPumpPin, !HIGH);        // Start Pump (inverse)
  startTimer1();                         // start Timer1 on 63s = cycleTimer(14) * 4,19424s
}

void stopPump(){
  digitalWrite(onPumpPin, !LOW);         // Stop Pump (inverse)
  cycle = cycleTimer;                    // Reset cycles
  TCNT1 = 0;                             // Reset Timer1
  stopTimer1();
}

void blockPump(){
  stopPump();
  digitalWrite(blockPumpPin, !HIGH);     // Disconnect Pump from power
  }

void unblockPump(){
  digitalWrite(blockPumpPin, !LOW);      // Connect Pump to power
}

void loop(){

//********** Control part - Commands and Buttons
  if(digitalRead(onPumpPin) && digitalRead(blockPumpPin) && (commandOnPump || digitalRead(buttonOnPumpPin))) {
    startPump();
  }
  if(!digitalRead(onPumpPin) && digitalRead(blockPumpPin) && (commandOffPump || digitalRead(buttonOffPumpPin))) {
    stopPump();
  }
  if(digitalRead(blockPumpPin) && (commandBlock || digitalRead(buttonBlockPin))) {
    blockPump();
  }
  if(!digitalRead(blockPumpPin) && (!commandBlock && !digitalRead(buttonBlockPin))) {
    unblockPump();
  }
}
```

Listing 4 - Forced Control Module

5.3.6 Fault Handling

Fault handling functions are divided between the two controllers. To allow for autonomous operation, the RCWPS controller detects only basic faults that can be handled without receiving additional information from the central controller. The MCC controller, in turn, processes faults whose detection requires the use of variable or calculated parameters.

The list of faults is specified in the chapter **Error! Reference source not found.**. As can be seen from the reserved capacity, there is still room for experimentation.

```
bool isError() {
  for(uint8_t i=0; i < 8; i++){
    if(codErrRCWPS[i] == true){
      alarmRCWPS = true;
      return true;
    }
  }
  alarmRCWPS = false;
  return false;
}

void loop(){
//****** Alarms
  for (int i = 0; i < 8; i++) {                    // reset array of alarms
  codErrRCWPS[i] = false;
  }
  if(!digitalRead(onPumpPin) && !digitalRead(voltagePumpPin)){     // If output onPump is ON (inverse) and no voltage
    codErrRCWPS[0] = true;                         // set error code
  } else {
    codErrRCWPS[0] = false;                        // reset error code
  }
  if(!digitalRead(blockPumpPin) && digitalRead(voltagePumpPin)){  // If output blockPump is ON (inverse) and voltage on
    codErrRCWPS[1] = true;                         // set error code
  } else {
    codErrRCWPS[1] = false;                        // set error code
  }
  // Control Alarm LED
  if(isError() || alarmMCC){
    digitalWrite(alarmPin, HIGH);
  } else {
    digitalWrite(alarmPin, LOW);
  }
}
```

Listing 5 - Fault Detection Module

5.4 MCC Controller

The ESP32 controller has various development toolkits available. The Arduino IDE development environment was used with installed modules from Espressif. The software is written in C++: https://github.com/Ochilnik/WPS/blob/master/MCC_Pump_8E.ino.

The same principles were used as in the previously described controller. Since the ESP32 has a different architecture, some functions have specific characteristics. A brief description of the code, broken down by its functional parts, is provided below.

5.4.1 Included Libraries

SPI.h - library for working with the SPI bus (the nRF2401 module is connected);

nRF24L01.h, RF24.h - libraries for working with the nRF2401 module;

Wire.h - library for working with the I2C bus (LCD and RTC modules);

LiquidCrystal_I2C.h - library for working with an LCD display via the I2C bus;

driver/timer.h - library for working with the ESP-IDF timer;

uRTCLib.h - library for working with the DS1307 via the I2C bus;

SD_MMC.h, FS.h – libraries for working with SD-MMC;

WiFi.h - library for working with WiFi;

5.4.2 Переменные, константы и объекты

```
#define TIMER_GROUP TIMER_GROUP_0     // Timer group
#define TIMER_IDX   TIMER_0           // Timer index
#define TIMER_DIVIDER 40000           // Prescaler (APB clock / 40000 = 2 kHz)
#define TIMER_ALARM_VALUE 240000      // Trigger value for a 120-second alarm (in 0.5 ms ticks)
#define NUM_AVG 10                    // Number of cycles for calculating the average current value
#define REQ_BUF_SZ   120              // size of buffer used to capture HTTP requests
#define ADRDAY 1                      // 1 byte address currentDay in I2C AT24C32 memory (addr: 1 - 56)
#define ADRNUMON 2                    // 1 byte address numOnPump in I2C AT24C32 memory (addr: 1 - 56)
#define ADRCOUNT 3                    // 2 bytes address countOnPump in I2C AT24C32 memory (addr: 1 - 56)
#define ADRCOUNTDAY 5                 // 2 bytes address vountDayPump in I2C AT24C32 memory (addr: 1 - 56)

LiquidCrystal_I2C lcd(0x27,16,2);     // Declare lcd object (I2C address = 0x27, columns = 16, rows = 2)
RF24 radio(13, 5);                    // Declare radio object. CE pin = 13, CSN pin = 5
String bits = "";                     // Declare string object 'bits'

byte rtcModel = URTCLIB_MODEL_DS1307; // Set RTC model
uRTCLib rtc;                          // Declare rtc object - clock

File webFile;                         // the web page file on the SD card
char HTTP_req[REQ_BUF_SZ] = {0};      // buffered HTTP request stored as null terminated string
char req_index = 0;                   // index into HTTP_req buffer

const char *ssid = "SSID";            // WiFi name
const char *password = "password";    // WiFi password
NetworkServer server(80);             // Declare network server object


const uint64_t addr_tx = 0xAABBCCDD22LL;  // connection address for transmission
const uint64_t addr_rx = 0xAABBCCDD11LL;  // connection address for reception
const uint8_t num_ch = 119;           // channel number
const float timeCycle = 4.19424;      // One cycle time is Arduino UNO counter

bool voltageOn;                       // Voltage is ON to pump
bool commandOnPump;                   // command to ON pump from RCWPS (Remote Controller for Water Pump Station)
bool commandOffPump;                  // command to OFF pump from RCWPS (Remote Controller for Water Pump Station)
bool commandBlock;                    // command to Block pump from RCWPS (Remote Controller for Water Pump Station)
bool stateAlarm, alarmMCC, alarmRCWPS, stateOnPump, prevStateOnPump, stateBlock // States signals from RCWPS
bool trigerBlock = 0, prevBlock = 0;  // trigger flags
bool tMenu, prevMenu;                 // touch button Menu, previos state button Menu
bool codErrMCC[8];                    // code of errors from MCC
bool codErrRCWPS[8];                  // code of errors from RCWPS
volatile bool backLight = false;      // Global state LCD light
volatile bool flagLight = false;      // Global flag state LCD light
volatile bool flagCount = false;      // Global flag enable counters from rtc
volatile bool flagPulse = false;      // Global flag is pulse from rtc.sq 1Hz
bool webOn, webOff, webBlock;         // command ON, OFF, Block from web client
bool pumpON, pumpOFF;                 // summ commands from MCC & WebClient

uint8_t data_tx[5] = {0};             // Array data to transmit
uint8_t data_rx[15] = {0};            // Array data to recieve
uint8_t data_tx_previos[5] = {0};     // Array data that transmit last time
uint8_t numOnPump;                    // number of pump starts
uint8_t currentDay;                   // today
uint8_t numScreen = 1;                // number current screen on Display
const uint8_t lcdScreens = 3;         // number screens lcd Display
uint16_t year;                        // current year
```

```
uint8_t month;                  // current month
uint8_t day;                    // current day
uint8_t hour;                   // current hour
uint8_t minute;                 // current minute
uint8_t dayHour;                // hours in day counter
uint8_t dayMinute;              // minutes in day counter
uint8_t daySecond;              // seconds in day counter
uint8_t tHour;                  // hours in last counter
uint8_t tMinute;                // minutes in last counter
uint8_t tSecond;                // seconds in last counter
uint8_t lowByte;                // Low byte
uint8_t highByte;               // High byte

uint16_t countOnPump;           // time last start
uint16_t countDayPump;          // pump operating time for the whole day
const uint16_t maxOnPump = 600;     // max time pump is working at once
const uint16_t maxDayPump = 3600;   // max time pump is working at day

int timeToOff = 0;              // Remaining time until pump shutdown
int humidity;                   // Humiduty from RCWPS (Remote Controller for Water Pump Station)
int pressure;

float currentRMS;               // RMS current value
float temperature;              // Temperatures from RCWPS (Remote Controller for Water Pump Station)
const float maxCurrent = 2.95;      // max RMS current value
const float minCurrent = 2.5;       // min RMS current value
const float maxTemp = 30;           // max temperatures
float avgCurrent;               // average currentRMS

// Pinout configuration
uint8_t buttonMenu = 4;     // button menu on is pressed
uint8_t buttonOnPumpPin = 27;   // button pump on is pressed
uint8_t buttonOffPumpPin = 32;  // button pump off is pressed
uint8_t buttonBlockPin = 33;    // triger blocking pump is on
uint8_t ledOnPumpPin = 16;      // led pump is on
uint8_t ledAlarmPin = 17;       // led alarm is on
uint8_t ledBlockPin = 25;       // led blocking pump is on
uint8_t buzzerPin = 26;         // buzzer alarm is on
uint8_t rtcSQPin = 35;          // rtc wave 1Hz input

void setup(){
// ******************************** Settings INPUT/OUTPUT ***********************
  //pinMode(buttonMenu, INPUT_PULLUP);      //  button menu
  pinMode(buttonOnPumpPin, INPUT_PULLUP);   // button pump on
  pinMode(buttonOffPumpPin, INPUT_PULLUP);  // button pump off
  pinMode(buttonBlockPin, INPUT_PULLUP);    // button blocking pump
  pinMode(rtcSQPin, INPUT);             // SQ wave from RTC
  pinMode(ledOnPumpPin, OUTPUT);            // led pump is on
  pinMode(ledAlarmPin, OUTPUT);             // led signal attention
  pinMode(ledBlockPin, OUTPUT);             // led blocking pump is on
  pinMode(buzzerPin, OUTPUT);               // sound signal attention
}
```

Listing 6 – Variables, Constants, and Objects Module

5.4.3 Packet Reception / Transmission

This is a familiar controller function. The settings are nearly identical to the previous controller, with the exception of the connection addresses for reception and transmission, which have been swapped. The commented-out parameters have been left in for debugging convenience.

```
void setup(){
// ***************************** Settings Radio Interface ***************************
  Serial.println("Reciver/Transmitter ON");
  radio.begin();                // initialize radio module
  delay(2000);
  //radio.setPayloadSize(7);      // Set static payload size in bytes (0-32)
  radio.setDataRate(RF24_250KBPS);  // data rate: RF24_1MBPS, RF24_2MBPS or RF24_250KBPS
```

```
  switch(radio.getDataRate()){                    // Check the set data rate radio.getDataRate()
    case RF24_1MBPS   : Serial.println("setDataRate  1 MBPS"); break; // If the received value is RF24_1MBPS
    case RF24_2MBPS   : Serial.println("setDataRate  2 MBPS"); break; // If the received value is RF24_2MBPS
    case RF24_250KBPS : Serial.println("setDataRate  250 KBPS"); break;
  }
  radio.setCRCLength(RF24_CRC_8);   // Set CRC size to 8 bit or 16 bit
  radio.setChannel(num_ch);         // set channel number
  radio.setAutoAck(true);           // enable auto-acknowledgment
  radio.setPALevel(RF24_PA_MAX);    // power amplifier: RF24_PA_MIN, RF24_PA_LOW, RF24_PA_HIGH or RF24_PA_MAX

  //-----------------------Reciever part-----------------------
  //radio.enableDynamicPayloads();  //enable dynamic payload support on the receiver
  radio.openReadingPipe(1, addr_rx); // open reading pipe
  //closeReadingPipe();             // Close a pipe that was previously opened for listening (data reception).
  //radio.enableAckPayload();       // allow sending data in response to an incoming signal
  radio.startListening();           // switch module to receiver mode

  //----------------------Transmitter part-------------------------
  radio.setRetries(5, 15);          // retry delay = (number+1) * 250 µs and number of retry attempts from 0 to 15
  radio.openWritingPipe(addr_tx);   // open writing pipe
  //radio.stopListening();          // switch module to transmitter mode
  //radio.powerUp();                // full power mode
  //radio.powerDown();              // low power consumption mode
}
void loop(){
//********************** Reciever part ******************************************88
  if(radio.available()){                    // If data is available in the buffer, then...
    radio.read(&data_rx, sizeof(data_rx));     // Read data from the buffer into the data_rx array

    //RX BYTE 0
    voltageOn = (data_rx[0] >> 0) & 0x01;       // Bit 0. Remote signal that voltage is on to Pump
    commandOnPump = (data_rx[0] >> 1) & 0x01;   // Bit 1. Remote Command to On Pump
    commandOffPump = (data_rx[0] >> 2) & 0x01;  // Bit 2. Remote Command to Off Pump
    commandBlock = (data_rx[0] >> 3) & 0x01;    // Bit 3. Remote Command to Block work Pump

    //RX BYTE 1
    alarmRCWPS = (data_rx[1] >> 0) & 0x01;      // Bit 0. State Alarm
    stateOnPump = (data_rx[1] >> 1) & 0x01;     // Bit 1. State On Relay to ON Pump
    stateBlock = (data_rx[1] >> 2) & 0x01;      // Bit 2. State On Relay to Block work Pump

    //RX BYTES 2 - 3. Motors current
    uint16_t current_raw = (data_rx[2] << 8) | data_rx[3]; // Combine two bytes
    currentRMS = current_raw / 100.0;              // Convert back to float

    //RX BYTES 4 - 5. Reserved for 15Ai - Waters pressure

    //RX BYTES 6 - 8. Temperature and humidity
    uint16_t temperature_raw = (data_rx[6] << 8) | data_rx[7]; // Combine two bytes
    temperature = temperature_raw / 10.0;              // Convert back to float
    humidity = data_rx[8];

    //RX BYTE 9. Array of errors codes from RCWPS
    for (int i = 0; i < 8; i++) {
      codErrRCWPS[i] = data_rx[9] & (1 << i); // Check the bit at position i and save to the array
    }

    //RX BYTE 10. Rest time to off pump
    timeToOff = static_cast<int>(timeCycle * data_rx[10]);
  }

//******************** Transmitter part ************************
  // Prepare data
  //TX BYTE 0
  data_tx[0] =  (trigerBlock << 3) | (pumpOFF << 2) | (pumpON << 1) | alarmMCC;
  //TX BYTE 1
  //TX BYTE 2
  //TX BYTE 3
  //TX BYTE 4

  // Transmit data if not equal previous
  if(!dataAreEqual(data_tx, data_tx_previos, sizeof(data_tx))) {
    radio.stopListening();                    // Stop listening, disable the receiver.
    radio.write(&data_tx, sizeof(data_tx));      // Send data from the data_tx array.
```

```
    // SerialPrint tx data. Generate bit string
    Serial.println("      TX");
    bits = "";
    for (int i = 7; i >= 0; i--) {
        bits += String((data_tx[0] >> i) & 1);  // Right shift and bitwise AND operation
    }
    Serial.println("      Inputs = " + bits);
    Serial.println();
    radio.startListening();                   // Start listening (enable receiver)
  }

  // Copy current station to previous station
  for(uint8_t i = 0; i < sizeof(data_tx); i++) {
    data_tx_previos[i] = data_tx[i];
  }
}
```

Listing 7 - Packet Reception / Transmission Module

In the main loop, the received information is first processed and decoded into variables. Then, the packet for transmission is prepared. The library's transmission function sends up to 32 bytes per packet (5 bytes are used in this project).

For efficient use of the available packet address space, variables of type BOOL are packed into bytes in groups.

The data packet is prepared during each operating cycle but is only transmitted if it differs from the packet generated in the previous cycle. To achieve this, after the packets are compared and the data is sent, the prepared packet is copied to a temporary array.

This approach significantly reduces radio frequency congestion and the controller's power consumption without compromising communication reliability.

5.4.4 Display

As mentioned earlier, parameters are shown on the display across several screens:

Screen 1: Counters and timers;

Screen 2: Measured parameter values;

Screen 3: Error messages.

Switching between them is done using the MENU button. On screen 3, the error messages cycle in a loop.

```
void setup(){
// ************************* Settings LCD on I2C *************************
  lcd.init();                    //  Initialize the LCD display
}

void loop(){
  // switch to around LCD Screens after MENU button pressing
  //if(!digitalRead(buttonMenu) && backLight && prevMenu){    // version for Menu button
  if(tMenu && backLight && prevMenu){                         // version for touch Menu
    numScreen++;
    if(numScreen > lcdScreens){
      numScreen = 1;
    }
  }
  //prevMenu = digitalRead(buttonMenu);
  prevMenu = tMenu;
toDisplay();
}
```

```
// function out information on LCD display. use global variables
void toDisplay() {
   // Previous state of variables
   static bool pv;         // voltageOn
   static bool par;        // alarmRCWPS
   static bool pam;        // alarmMCC
   static bool psop;       // stateOnPump
   static bool psb;        // stateBlock
   static uint8_t pnd;     // number screen on display
   static uint8_t phour;     // Hour
   static uint8_t pminute;      // Minute
   static uint8_t pnop;    // numOnPump
   static uint8_t ph;      // humidity
   static uint16_t pcop;   // countOnPump
   static uint16_t pcdp;   // countDayPump
   static uint8_t ptto;    // timeToOff
   static float pi;        // currentRMS
   static float pt;        // temperatures

   static uint8_t iNmsg;   // number error message to screen
   static uint8_t iTshow;  // counter times to show error message

   char* errMsgRCWPS[8] = {                    // mesages of errors from MCC
     " RelayON noVolt",
     " BlockON VoltON",
     "",
     "",
     "",
     "",
     "",
     "",
     ""
   };

   char errMsgMCC[8][16];  // 8 messages, each up to 16 characters long
   // Convert messages to char arrays
   snprintf(errMsgMCC[0], sizeof(errMsgMCC[0]), "Last ON > %dayMinute", maxOnPump / 60);
   snprintf(errMsgMCC[1], sizeof(errMsgMCC[1]), "Day ON > %dayMinute", maxDayPump / 60);
   snprintf(errMsgMCC[2], sizeof(errMsgMCC[2]), "I=%.2fA > %.2f", avgCurrent, maxCurrent);
   snprintf(errMsgMCC[3], sizeof(errMsgMCC[3]), "I=%.2fA < %.2f", avgCurrent, minCurrent);
   snprintf(errMsgMCC[4], sizeof(errMsgMCC[4]), "T=%.2fC > %.2f", temperature, maxTemp);
   strcpy(errMsgMCC[5], "");
   strcpy(errMsgMCC[6], "");
   strcpy(errMsgMCC[7], "");

  // redraw display only if parameters change
  if(par != alarmRCWPS || pam != alarmMCC || pnd != numScreen ||
    phour != hour || pminute != minute || pnop != numOnPump || ph != humidity ||
    pcop != countOnPump || pcdp != countDayPump ||
    ptto != timeToOff || pi != currentRMS || pt != temperature) {

    if(numScreen == 1) {
       // 1st string - Clock
       lcd.clear();                    //  Clear the display
       lcd.setCursor(0, 0);            //  Set cursor to position (column 0, row 0)
       lcdPrintZeros(String(hour), 0, 2, 0);   //  lcd.print(u == 1 ? "ON" : "OFF");
       lcd.setCursor(2, 0);
       lcd.print(":");
       lcdPrintZeros(String(minute), 3, 2, 0);
       // 1st string - Timer Day
       lcd.setCursor(6, 0);
       lcd.print("D");
       lcdPrintZeros(String(dayHour), 7, 2, 0);   // Hours from countOnDay
       lcd.setCursor(9, 0);
       lcd.print("h");
       lcdPrintZeros(String(dayMinute), 10, 2, 0);  // Minutes from countOnDay
       lcd.setCursor(12, 0);
       lcd.print("m");
       lcdPrintZeros(String(daySecond), 13, 2, 0);  // Seconds from countOnDay
       lcd.setCursor(15, 0);
       lcd.print("s");
       // 2nd string - Rest Time Pump is On
       lcd.setCursor(0, 1);
```

```
      lcd.print("R");
      lcdPrintZeros(String(timeToOff), 1, 3, 1);  // Rest time pump is on
      lcd.setCursor(4, 1);
      lcd.print("s");
      // 2nd string - number of starts
      lcd.setCursor(6, 1);
      lcd.print("N");
      lcd.setCursor(7, 1);
      lcd.print(numOnPump);
      // 2nd string - Time the last of pump start
      lcdPrintZeros(String(tMinute), 10, 2, 1);  // Time the last of pump start
      lcd.setCursor(12, 1);
      lcd.print("m");
      lcdPrintZeros(String(tSecond), 13, 2, 1);
      lcd.setCursor(15, 1);
      lcd.print("s");
  }
  if(numScreen == 2) {
      // 1st string - Clock
      lcd.clear();                    //  Clear the display
      lcd.setCursor(0, 0);            //  Set cursor to position (column 0, row 0)
      lcdPrintZeros(String(hour), 0, 2, 0);   //  lcd.print(u == 1 ? "ON" : "OFF");
      lcd.setCursor(2, 0);
      lcd.print(":");
      lcdPrintZeros(String(minute), 3, 2, 0);
      // 1st string - Temperature
      lcdPrintZeros(String(temperature), 6, 4, 0);  // Temperatures
      lcd.setCursor(10, 0);
      lcd.print("C");
      lcdPrintZeros(String(humidity), 13, 2, 0); // Humidity
      lcd.setCursor(15, 0);
      lcd.print("%");
      // 2nd string - RMS Current
      lcd.setCursor(0, 1);
      lcd.print("I");
      lcd.setCursor(1, 1);
      lcd.print(currentRMS);
      // 2nd string - number of starts
      lcd.setCursor(6, 1);
      lcd.print("N");
      lcd.setCursor(7, 1);
      lcd.print(numOnPump);         // Number of pump starts
      // 2nd string - Time the last of pump start
      lcdPrintZeros(String(tMinute), 10, 2, 1);  // Time the last of pump start
      lcd.setCursor(12, 1);
      lcd.print("m");
      lcdPrintZeros(String(tSecond), 13, 2, 1);
      lcd.setCursor(15, 1);
      lcd.print("s");
  }
}
  if(numScreen == 3) {
      // output error message RCWPS on 1st string, MCC on 2nd string every 4 loop
      if(iTshow % 4 == 0){
        lcd.clear();                    //  Clear the display
        lcd.setCursor(0, 0);
        lcd.print(iNmsg);
        if(codErrRCWPS[iNmsg] == true){lcd.print(errMsgRCWPS[iNmsg]);} else {lcd.print(" RCWPS is OK");}
        lcd.setCursor(0, 1);
        if(codErrMCC[iNmsg] == true){lcd.print(errMsgMCC[iNmsg]);} else {lcd.print(" MCC is OK");}
        iNmsg = (iNmsg + 1) % 5;            // counter for error message number, resets to zero every 5
      }
      iTshow++;                         // counter for the number of times the error message is shown
  }

 pv = voltageOn;
 par = alarmRCWPS;
 pam = alarmMCC;
 psop = stateOnPump;
 psb = stateBlock;
 pnd = numScreen;
 phour = hour;
 pminute = minute;
```

```
  pnop = numOnPump;
  ph = humidity;
  pcop = countOnPump;
  pcdp = countDayPump;
  ptto = timeToOff;
  pi = currentRMS;
  pt = temperature;

} // end toDisplay()

// Function to print text with leading zeros
void lcdPrintZeros(String value, uint8_t colB, uint8_t width, uint8_t row) {
    String formattedValue = value;

    // Add leading zeros if the string length is less than the required width
    while (formattedValue.length() < width) {
        formattedValue = "0" + formattedValue;
    }

    // Limit the string length (in case of errors)
    if (formattedValue.length() > width) {
        formattedValue = formattedValue.substring(0, width);
    }
    lcd.setCursor(colB, row);
    lcd.print(formattedValue);
} // end lcdPrintZeros()
```

Listing 8 - Display Module

5.4.5 Display Backlight Timer

To manage the display backlight duration, hardware timer 0 of group 0 of the microcontroller is used, along with standard timer control libraries.

The timer triggers based on a set value:

- APB clock frequency divider of 40000, which results in 80MHz/40000=2kHz;
- One clock cycle duration is 1 / 2 kHz = 0.0005 seconds;
- The timer runs for 240000 * 0.0005 = 120 seconds before triggering.

In the ESP32 controller, the timer settings allow for organizing long time delays. Therefore, it can achieve a 2-minute delay in a single cycle.

The timer and the backlight are activated by touching the MENU button and are deactivated within the timer's subroutine after the countdown is complete.

```
// Turn off display backlight. Timer interrupt service routine (ISR). IRAM_ATTR means the function is loaded into RAM
void IRAM_ATTR onTimer(void* arg){
  timer_group_clr_intr_status_in_isr(TIMER_GROUP, TIMER_IDX); // Clear the timer's interrupt flag
  // code to handle the interrupt
  backLight = false;              //  Turn off the LCD backlight
  flagLight = true;              // flag indicate that backLight is changed
  timer_pause(TIMER_GROUP, TIMER_IDX);  // Stop the timer
  timer_set_alarm(TIMER_GROUP, TIMER_IDX, TIMER_ALARM_EN);  // Once triggered, the alarm is disabled automatically
and needs to be re-enabled to trigger again.
}

void setup(){
// ********************* Settings backlight timer LCD **********************
  timer_config_t timeconf = {
    .alarm_en = TIMER_ALARM_EN,
    .counter_en = TIMER_PAUSE,
    .intr_type = TIMER_INTR_LEVEL,
    .counter_dir = TIMER_COUNT_UP,
    .auto_reload = TIMER_AUTORELOAD_EN,
    .divider = TIMER_DIVIDER
  };
```

```
  timer_init(TIMER_GROUP, TIMER_IDX, &timeconf);
  timer_set_alarm_value(TIMER_GROUP, TIMER_IDX, TIMER_ALARM_VALUE); // Set the alarm value
  timer_enable_intr(TIMER_GROUP, TIMER_IDX);  // Enable interrupts
  timer_isr_register(TIMER_GROUP, TIMER_IDX, onTimer, NULL, ESP_INTR_FLAG_IRAM, NULL); // Attach the interrupt
handler
}

void loop(){
//******************************** LCD ***************************************
  // Backlight LCD after MENU button pressing
    if(touchRead(buttonMenu) <= 25) {tMenu = true;} else {tMenu = false;}
    if(tMenu && !backLight && !flagLight){
    lcd.setBacklight(1);                //  Turn on the LCD backlight
    backLight = true;
    timer_start(TIMER_GROUP, TIMER_IDX);  // Start the timer
  }
  if (!tMenu && !backLight && flagLight) {
    lcd.setBacklight(0);                //  Turn off the LCD backlight
    flagLight = false;
  }

}
```

Listing 9 - Timer Module

5.4.6 Real-Time Clock

The capabilities of the pump station control system are significantly expanded by using a real-time clock (RTC) module with an independent power supply. In addition to displaying the current time, it allows for tracking operating hours on a daily basis, maintaining weekly records, and much more.

Furthermore, a 1 Hz signal from a separate output of the module is used for counting the operating time.

The accuracy of the clock depends on the precision of the installed quartz crystal. Time setting is implemented via a web interface, and the corresponding software logic is located in the **Error! Reference source not found.**module.

```
void setup(){
// ********************** Settings Real Time clock DS1307 ********************
  URTCLIB_WIRE.begin();
  rtc.set_rtc_address(0x68);
            rtc.set_model(rtcModel);  // set RTC Model
            rtc.refresh();  // refresh data from RTC HW in RTC class object so flags like rtc.lostPower(), rtc.getEOSCFlag(), etc, can
get populated
  // Only use once, then disable
  // RTCLib::set(byte second, byte minute, byte hour (0-23:24-hour mode only), byte dayOfWeek (Sun = 1, Sat = 7), byte
dayOfMonth (1-12), byte month, byte year)
  rtc.sqwgSetMode(URTCLIB_SQWG_1H); // Setting SQWG/INT output
  delay(10);
}

void loop(){
// refresh and read data from RTC HW in RTC class object so flags like rtc.lostPower(), rtc.getEOSCFlag(), etc, can get
populated
  rtc.refresh();
  year = rtc.year();
  month = rtc.month();
  day = rtc.day();
  hour = rtc.hour();
  minute = rtc.minute();
  dayHour = countDayPump/60/60;
  dayMinute = (countDayPump - dayHour * 60)/60;
  daySecond = countDayPump%60;
  tHour = countOnPump/60/60;
  tMinute = (countOnPump - tHour * 60)/60;
```

```
  tSecond = countOnPump%60;
}
```

Listing 10 - Real-Time Clock Module

5.4.7 Operating Time Counters

To count the operating time, a 1 Hz signal from a separate output of the RTC module is used. This signal triggers a hardware interrupt, and an event handler subroutine is executed. The counting process is controlled by enabling and disabling this interrupt.

A key characteristic of hardware interrupt subroutines is the need for them to execute quickly, which means they must contain a minimal amount of code. Therefore, when an interrupt occurs, it only sets a flag and increments a counter; the rest of the algorithm is executed in the main program.

The following counters are implemented:

- A counter for the duration of the last "ON" cycle;
- A counter for the total operating time for the current day;
- A counter for the number of "ON" cycles for the current day.

In addition to the clock, the RTC module is equipped with AT24C32 flash memory (cell addresses from 1 to 56). Functions for accessing this memory are included in the rtc library. In the Real-Time Clock module, the previously saved counter values are read. The writing of counter values is performed in this module.

```
// function interrupt counters operating time from external wave 1Hz
void IRAM_ATTR counterOn(){
  countOnPump++;
  flagPulse = true;
}

void setup(){
// ************************* Read stored data *********************************
  currentDay = rtc.ramRead(ADRDAY);     // Read currentDay in I2C AT24C32 memory (addr: 1 - 56)
  numOnPump = rtc.ramRead(ADRNUMON);     // Read numOnPump in I2C AT24C32 memory (addr: 1 - 56)
  countOnPump = (rtc.ramRead(ADRCOUNT+1) << 8) | rtc.ramRead(ADRCOUNT);  // Read countOnPump in I2C AT24C32
memory (addr: 1 - 56)
  countDayPump = (rtc.ramRead(ADRCOUNTDAY+1) << 8) | rtc.ramRead(ADRCOUNTDAY);    // Read countDayPump in I2C
AT24C32 memory (addr: 1 - 56)
}

void loop(){
// ************* Counters working time ****************************
  // ON interrupt from 1Hz signal
  if(voltageOn && !prevStateOnPump && !flagCount){
    attachInterrupt(digitalPinToInterrupt(rtcSQPin), counterOn, FALLING);
    flagCount = true;
    countOnPump = 0;
    numOnPump++;
    prevStateOnPump = true;
    rtc.ramWrite(ADRNUMON, numOnPump);   // Write numOnPump in I2C AT24C32 memory (addr: 1 - 56)
  }
  // OFF interrupt from 1Hz signal
  if(!voltageOn && !stateOnPump && prevStateOnPump && flagCount){
    detachInterrupt(digitalPinToInterrupt(rtcSQPin));
    flagCount = false;
    prevStateOnPump = false;
    lowByte = countOnPump & 0xFF;          // Low Byte
    highByte = (countOnPump >> 8) & 0xFF;   // High Byte
    rtc.ramWrite(ADRCOUNT, lowByte);       // Write countOnPump in I2C AT24C32 memory (addr: 1 - 56)
    rtc.ramWrite(ADRCOUNT+1, highByte);     // Write countOnPump in I2C AT24C32 memory (addr: 1 - 56)
```

```
    lowByte = countDayPump & 0xFF;          // Low Byte
    highByte = (countDayPump >> 8) & 0xFF;  // High Byte
    rtc.ramWrite(ADRCOUNTDAY, lowByte);          // Write countDayPump in I2C AT24C32 memory (addr: 1 - 56)
    rtc.ramWrite(ADRCOUNTDAY+1, highByte);       // Write countDayPump in I2C AT24C32 memory (addr: 1 - 56)
  }
  // continue interrupt handling 1Hz counter
  if(flagPulse){
    if(currentDay == rtc.day()){
      countDayPump++;
    } else {
      countDayPump = 0;
      numOnPump = 0;
      currentDay = rtc.day();
      rtc.ramWrite(ADRDAY, currentDay);   // Reset numOnPump in I2C AT24C32 memory (addr: 1 - 56)
      rtc.ramWrite(ADRNUMON, 0);   // Reset numOnPump in I2C AT24C32 memory (addr: 1 - 56)
      rtc.ramWrite(ADRCOUNTDAY, 0);    // Reset countDayPump in I2C AT24C32 memory (addr: 1 - 56)
      rtc.ramWrite(ADRCOUNTDAY+1, 0);    // Reset countDayPump in I2C AT24C32 memory (addr: 1 - 56)
    }
    flagPulse = false;
  }
}
```

Listing 11 - Counter Module

5.4.8 Control and Indication

The control part of the program identifies a command from the buttons or the web interface and sets the corresponding variable for subsequent transmission via radio to the peripheral controller for execution.

The block command operates in a toggle (trigger) mode. For this controller, a software trigger is implemented.

```
void loop(){
//********************** Control part - LED *********************************
  // Control LED pump state. Green
  if(voltageOn){
    digitalWrite(ledOnPumpPin, HIGH);
  } else {
    digitalWrite(ledOnPumpPin, LOW);
  }
  // Control LED pump block state. Red
  if(stateBlock){
    digitalWrite(ledBlockPin, HIGH);
  } else {
    digitalWrite(ledBlockPin, LOW);
  }
  // Control LED alarm state. Yellow
  if(stateAlarm){
    digitalWrite(ledAlarmPin, HIGH);
  } else {
    digitalWrite(ledAlarmPin, LOW);
  }

//******************* Control part - Buttons  ***************************
  // command ON Pump
  pumpON = !digitalRead(buttonOnPumpPin) || webOn;
  // command OFF Pump
  pumpOFF = !digitalRead(buttonOffPumpPin) || webOff;

  // Triger that switch by pressing BLOCK button
  if((!digitalRead(buttonBlockPin) == true && !digitalRead(buttonBlockPin) != prevBlock && trigerBlock == false) ||
    (webBlock == true && webBlock != prevBlock && trigerBlock == false)) {
    trigerBlock = true;
  } else if((!digitalRead(buttonBlockPin) == true && !digitalRead(buttonBlockPin) != prevBlock && trigerBlock == true) ||
    (webBlock == true && webBlock != prevBlock && trigerBlock == true)){
    trigerBlock = false;
  }
```

```
  prevBlock = !digitalRead(buttonBlockPin) || webBlock;
}
```

Listing 12 - Control and Indication Module

5.4.9 Fault Handling

Fault handling functions are divided between the two controllers. The MCC controller processes faults whose detection requires the use of variable or calculated parameters.

The list of faults is specified in the chapter "**Error! Reference source not found.**."

```
void loop(){
//********************* Alarms **********************************
  stateAlarm = alarmMCC || alarmRCWPS;
  // Check max time working pump
  if(countOnPump > maxOnPump){
    codErrMCC[0] = true;
  } else {
    codErrMCC[0] = false;
  }
  // Check max day time working pump
  if(countDayPump > maxDayPump){
    codErrMCC[1] = true;
  } else {
    codErrMCC[1] = false;
  }

  // Calculate average current
  avgCurrent = calcAverage(currentRMS);
  // Check max working rms current
  if(avgCurrent > maxCurrent){
    codErrMCC[2] = true;
  } else {
    codErrMCC[2] = false;
  }
  // Check min working rms curren t
  if(voltageOn && avgCurrent < minCurrent){
    codErrMCC[3] = true;
  } else {
    codErrMCC[3] = false;
  }
  // Check max working temperatures
  if(temperature > maxTemp){
    codErrMCC[4] = true;
  } else {
    codErrMCC[4] = false;
  }

  // Set flag alarmMCC if in array codErrMCC error
  alarmMCC = false;
  for (uint8_t i = 0; i < 8; i++) {
    alarmMCC = alarmMCC || codErrMCC[i];  // Perform a logical OR
    if (alarmMCC) break;                  // If the result is already true, the loop can be broken
  }
}

// function calculate average value from several (NUM_AVG)
float calcAverage(float current) {
  static float values[NUM_AVG] = {0};       // Buffer to store the last values
  static int index = 0;                  // Current index in the buffer
  static int count = 0;                  // Number of filled elements (up to 10)
  //int current = round(10 * val);          // convert to int

  // Write the new value to the buffer
  values[index] = current;
  index = (index + 1) % NUM_AVG; // Move the index circularly
```

```
  // Increment the counter of filled elements, but not more than NUM_AVG
  if (count < NUM_AVG) {
    count++;
  }

  // Calculate the average value
  float sum = 0;
  for (int i = 0; i < count; i++) {
    sum += values[i];
  }
  // static_cast<float>(sum) / count / 10;  // convert to float
  return sum / count;
} // end calcAverage()
```

Listing 13 - Fault Detection Module

5.4.10 SD-MMC Memory

In this project, an SD-MMC memory card is used to store the web interface pages. The SD-MMC module includes the initialization section, while reading from the memory card is performed in the **Error! Reference source not found.**module.

For future development of the project, it is also planned to store data here over an extended period for graphing and analysis.

```
void setup(){
// ****************************** Settings SD-MMC card ************************************
  if (!SD_MMC.begin("/sdcard", true)) {
    Serial.println("Card Mount Failed");
    return;
  }
  uint8_t cardType = SD_MMC.cardType();
  if (cardType == CARD_NONE) {
    Serial.println("No SD_MMC card attached");    // !!!!!!!!!!!!!!!
    return;
  }
  Serial.print("SD_MMC Card Type: ");
  if (cardType == CARD_MMC) {
    Serial.println("MMC");
  } else if (cardType == CARD_SD) {
    Serial.println("SDSC");
  } else if (cardType == CARD_SDayHourC) {
    Serial.println("SDayHourC");
  } else {
    Serial.println("UNKNOWN");
  }
  Serial.printf("Total space: %lluMB\n", SD_MMC.totalBytes() / (1024 * 1024));
  Serial.printf("Used space: %lluMB\n", SD_MMC.usedBytes() / (1024 * 1024));
  // check for index.html file
  if (!SD_MMC.exists("/index.html")) {
      Serial.println("ERROR - Can't find index.html file!");
      return;  // can't find index file
  }
  delay(10);
}
```

Listing 14 - SD-MMC Module

5.4.11 Wi-Fi

In this project, a Wi-Fi connection to the local network is used to provide access to the web interface page of the pump station control system. The Wi-Fi module includes the initialization section, and its objects are accessed within the **Error! Reference source not found.** module.

```
void setup(){
// ************************** Settings Wi-Fi ***********************************
  Serial.println();
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.begin(ssid, password);              //  Network.begin(mac, ip);  // initialize Network device
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi connected.");

  // Print connections parameters
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
  Serial.print("Hostname: ");
  Serial.println(WiFi.getHostname());
  Serial.print("ESP Mac Address: ");
  Serial.println(WiFi.macAddress());
  Serial.print("Subnet Mask: ");
  Serial.println(WiFi.subnetMinuteask());
  Serial.print("Gateway IP: ");
  Serial.println(WiFi.gatewayIP());
  Serial.print("DNS: ");
  Serial.println(WiFi.dnsIP());

  server.begin();                          // start to listen for clients
}
```

Listing 15 - Wi-Fi module

5.4.12 Web Server

In this project, a significant portion of the program is dedicated to organizing the web server and handling information exchange.

When a client connects, the index.html file, read from the memory card, is transmitted to it. This file contains the CSS styling and a JavaScript program for processing parameter requests using the XMLHttpRequest standard.

The webCommand subroutine processes the START, STOP, BLOCK_WORK, and Set_Time commands received from the client in a GET request.

The XML_response subroutine prepares and transmits data to the client.

The setTimeFromHttp subroutine sets the real-time clock time based on the time on the client's device.

```
void setup(){
NetworkClient client = server.accept();       // try to get client

  if (client) {                             // got client?
      boolean currentLineIsBlank = true;
      while (client.connected()) {
        if (client.available()) {           // client data available to read
          char c = client.read();           // read 1 byte (character) from client
          // limit the size of the stored received HTTP request
          // buffer first part of HTTP request in HTTP_req array (string)
          // leave last element in array as 0 to null terminate string (REQ_BUF_SZ - 1)
          if (req_index < (REQ_BUF_SZ - 1)) {
            HTTP_req[req_index] = c;        // save HTTP request character
            req_index++;
          }
          // last line of client request is blank and ends with \n
          // respond to client only after last line received
```

```
            if (c == '\n' && currentLineIsBlank) {
               // send a standard http response header
               client.println("HTTP/1.1 200 OK");
               // remainder of header follows below, depending on if
               // web page or XML page is requested
               // Ajax request - send XML file
               if (StrContains(HTTP_req, "ajax_inputs")) {
                  // send rest of HTTP header
                  client.println("Content-Type: text/xml");
                  client.println("Connection: keep-alive");
                  client.println();
                  webCommand();
                  XML_response(client);                    // send XML file containing input states
               } else {                                    // web page request
                  // send rest of HTTP header
                  client.println("Content-Type: text/html");
                  client.println("Connection: keep-alive");
                  client.println();
                  // send web page
                  webFile = SD_MMC.open("/index.html");           // open web page file
                  if (webFile) {
                     while(webFile.available()) {
                        client.write(webFile.read());       // send web page to client
                     }
                     webFile.close();
                  }
               }
               // display received HTTP request on serial port
               Serial.print(HTTP_req);
               // reset buffer index and all buffer elements to 0
               req_index = 0;
               StrClear(HTTP_req, REQ_BUF_SZ);
               break;
            }
            // every line of text received from the client ends with \r\n
            if (c == '\n') {
               // last character on line of received text
               // starting new line with next character read
               currentLineIsBlank = true;
            } else if (c != '\r') {
               // a text character was received from client
               currentLineIsBlank = false;
            }
         } // end if (client.available())
      } // end while (client.connected())
      delay(1);      // give the web browser time to receive the data
      client.stop(); // close the connection
   } // end if (client)
}

// Resolve commands from web client
void webCommand(void){
  // Start
    if (StrContains(HTTP_req, "START")) {
      webOn = true;
    } else {
      webOn = false;
    }
  // Stop
    if (StrContains(HTTP_req, "STOP")) {
      webOff = true;
    } else {
      webOff = false;
    }
  // Block
    if (StrContains(HTTP_req, "BLOCK_WORK")) {
      webBlock = true;
    } else {
      webBlock = false;
    }
  // Set Time
    if (StrContains(HTTP_req, "second=")) {
      setTimeFromHttp(HTTP_req);
```

```
    }
} // end webCommand()

// send the XML file with analog values, switch status
void XML_response(NetworkClient cl){
    cl.println("<?xml version=\"1.0\" ?>");
    cl.println("<root>");

    cl.print("<data>");
    cl.print(addZero(String(day), 2) + "-" + addZero(String(month), 2) + "-20" + addZero(String(year), 2));
    cl.println("</data>");

    cl.print("<time>");
    cl.print(addZero(String(hour), 2) + ":" + addZero(String(minute), 2));
    cl.println("</time>");

    cl.print("<temperature>");
    cl.print(String(temperature));
    cl.println("</temperature>");

    cl.print("<humidity>");
    cl.print(String(humidity));
    cl.println("</humidity>");

    cl.print("<voltage>");
    cl.print(voltageOn == true ? "ON" : "OFF");
    cl.println("</voltage>");

    cl.print("<rmsCurrent>");
    cl.print(String(currentRMS));
    cl.println("</rmsCurrent>");

    cl.print("<pressure>");
    cl.print(String(pressure));
    cl.println("</pressure>");

    cl.print("<lastOnTime>");
    cl.print(addZero(String(tHour), 2) + "h " + addZero(String(tMinute), 2) + "m " + addZero(String(tSecond), 2) + "s");
    cl.println("</lastOnTime>");

    cl.print("<timePerDay>");
    cl.print(addZero(String(dayHour), 2) + "h " + addZero(String(dayMinute), 2) + "m " + addZero(String(daySecond), 2) + "s");
    cl.println("</timePerDay>");

    cl.print("<startsPerDay>");
    cl.print(String(numOnPump));
    cl.println("</startsPerDay>");

    cl.print("<timeToOff>");
    cl.print(addZero(String(timeToOff), 3) + "s");
    cl.println("</timeToOff>");

    cl.print("<block>");
    cl.print(String(stateBlock == true ? "ON" : "OFF"));
    cl.println("</block>");

    cl.print("<alarm>");
    cl.print(String(stateAlarm == true ? "ON" : "OFF"));
    cl.println("</alarm>");

    cl.print("<relay>");
    cl.print(String(stateOnPump == true ? "ON" : "OFF"));
    cl.println("</relay>");

    cl.println("</root>");
} // end XML_response()

// Function set time to DS1307 from client http request string
void setTimeFromHttp(String http_req) {
  uint8_t second;
  uint8_t minute;
  uint8_t hour;
  uint8_t dayOfWeek;
```

```
  uint8_t dayOfMonth;
  uint8_t month;
  uint8_t year;

int paramStart = http_req.indexOf('?');
  if (paramStart == -1) {
    Serial.println("Error: query string not found!");
    return;
  }

  // Extract parameters
  String params = http_req.substring(paramStart + 1);
  int paramEnd = params.indexOf(' ');
  if (paramEnd != -1) {
    params = params.substring(0, paramEnd);
  }

  // Split parameters by '&'
  while (params.length() > 0) {
    int eqIndex = params.indexOf('=');
    int ampIndex = params.indexOf('&');

    if (eqIndex != -1) {
      String key = params.substring(0, eqIndex);
      String value = (ampIndex != -1) ? params.substring(eqIndex + 1, ampIndex) : params.substring(eqIndex + 1);

      if (key == "second") second = value.toInt();
      else if (key == "minute") minute = value.toInt();
      else if (key == "hour") hour = value.toInt();
      else if (key == "dayOfWeek") dayOfWeek = value.toInt();
      else if (key == "dayOfMonth") dayOfMonth = value.toInt();
      else if (key == "month") month = value.toInt();
      else if (key == "year") year = value.toInt();

      if (ampIndex == -1) break; // If there are no more parameters, exit the loop
      params = params.substring(ampIndex + 1); // Remove the processed parameter
    } else {
      break;
    }
  }

  // Set the time on the DS1307 module
  // RTCLib::set(byte second, byte minute, byte hour (0-23:24-hour mode only), byte dayOfWeek (Sun = 1, Sat = 7), byte
dayOfMonth (1-12), byte month, byte year)
  rtc.set(second, minute, hour, dayOfWeek, dayOfMonth, month, year);
  Serial.println("Time set successfully!");
  Serial.println();
} // end setTimeFromHttp()

// Function to pad text with zeros
String addZero(String value, uint8_t width) {
    String formattedValue = value;

    // Add leading zeros if the string length is less than the required width
    while (formattedValue.length() < width) {
        formattedValue = "0" + formattedValue;
    }

    // Limit the string length (in case of errors)
    if (formattedValue.length() > width) {
        formattedValue = formattedValue.substring(0, width);
    }
    return formattedValue;
} // end addZero()

// sets every element of str to 0 (clears array)
void StrClear(char *str, char length)
{
    for (int i = 0; i < length; i++) {
        str[i] = 0;
    }
}
// searches for the string sfind in the string str
```

```
// returns 1 if string found
// returns 0 if string not found
char StrContains(const char *str, const char *sfind)
{
   char found = 0;
   char index = 0;
   char len;

   len = strlen(str);

   if (strlen(sfind) > len) {
      return 0;
   }
   while (index < len) {
      if (str[index] == sfind[found]) {
         found++;
         if (strlen(sfind) == found) {
            return 1;
         }
      }
      else {
         found = 0;
      }
      index++;
   }
   return 0;
} // end StrContains()
```

Listing 16 - Web Server Module

5.4.13 Web Client

After connecting to the server, the client receives the html page https://github.com/Ochilnik/WPS/blob/master/index.html, which contains a JavaScript AJAX request script for retrieving data from the server.

The purpose of this script is the online updating of individual parameters, changing the layout, etc., on an already open page without a full redraw. The script receives system parameters, which are updated once per second, and transmits the specified commands. All communication occurs through the web interface in the client's browser.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Water pumping and storage station</title>

   <script>
      var data = "";
      var time = "";
      var temperature = "";
      var humidity = "";

      var voltage = "";
      var rmsCurrent = "";
      var pressure = "";

      var lastOnTime = "";
      var timePerDay = "";
      var startsPerDay = "";
      var timeToOff = "";

      var block = "";
      var alarm = "";
      var relay = "";
```

```
    var strCommand = "";

    // AJAX request to get data from the server
    function getDataFromServer() {
        var nocache = "&nocache=" + Math.random() * 1000000;
        var request = new XMLHttpRequest();
        request.onreadystatechange = function() {
            if (this.readyState == 4) {
                if (this.status == 200) {
                    if (this.responseXML != null) {
                        // Get data from the server
                        data = this.responseXML.getElementsByTagName('data')[0].childNodes[0].nodeValue;
                        time = this.responseXML.getElementsByTagName('time')[0].childNodes[0].nodeValue;
                        temperature = this.responseXML.getElementsByTagName('temperature')[0].childNodes[0].nodeValue;
                        humidity = this.responseXML.getElementsByTagName('humidity')[0].childNodes[0].nodeValue;

                        voltage = this.responseXML.getElementsByTagName('voltage')[0].childNodes[0].nodeValue;
                        rmsCurrent = this.responseXML.getElementsByTagName('rmsCurrent')[0].childNodes[0].nodeValue;
                        pressure = this.responseXML.getElementsByTagName('pressure')[0].childNodes[0].nodeValue;

                        lastOnTime = this.responseXML.getElementsByTagName('lastOnTime')[0].childNodes[0].nodeValue;
                        timePerDay = this.responseXML.getElementsByTagName('timePerDay')[0].childNodes[0].nodeValue;
                        startsPerDay = this.responseXML.getElementsByTagName('startsPerDay')[0].childNodes[0].nodeValue;
                        timeToOff = this.responseXML.getElementsByTagName('timeToOff')[0].childNodes[0].nodeValue;

                        block = this.responseXML.getElementsByTagName('block')[0].childNodes[0].nodeValue;
                        alarm = this.responseXML.getElementsByTagName('alarm')[0].childNodes[0].nodeValue;
                        relay = this.responseXML.getElementsByTagName('relay')[0].childNodes[0].nodeValue;

                        // Update elements on the page
                        document.getElementById("data").textContent = "Data: " + data;
                        document.getElementById("time").textContent = "Time: " + time;
                        document.getElementById("temperature").textContent = "Temperature: " + temperature + "°C";
                        document.getElementById("humidity").textContent = "Humidity: " + humidity + "%";

                        document.getElementById("voltage").value = voltage;
                        document.getElementById("voltage").style.backgroundColor = voltage === "ON" ? "#00FF00" : "#FFFFFF"; //
Green if ON, otherwise white
                        document.getElementById("rms").value = rmsCurrent;
                        document.getElementById("pressure").value = pressure;

                        document.getElementById("last-time").value = lastOnTime;
                        document.getElementById("time-per-day").value = timePerDay;
                        document.getElementById("starts-per-day").value = startsPerDay;
                        document.getElementById("time-to-off").textContent = timeToOff;

                        // Update button colors
                        document.getElementById("start-btn").style.backgroundColor = relay === "ON" ? "#00FF00" : "#C2DFFF";
                        document.getElementById("stop-btn").style.backgroundColor = alarm === "ON" ? "yellow" : "#C2DFFF";
                        document.getElementById("block-btn").style.backgroundColor = block === "ON" ? "red" : "#C2DFFF";
                    }
                }
            }
        };
        // Send request to the server
        request.open("GET", "ajax_inputs?" + strCommand + nocache, true);
        request.send(null);
        setTimeout(getDataFromServer, 1000); // Repeat the request every second
    }

    // PRESS BUTTONS
    function onStartClick() {
        strCommand = "action=START";
        getDataFromServer();
        strCommand = "";
    }
    function onStopClick() {
        strCommand = "action=STOP";
        getDataFromServer();
        strCommand = "";
    }
    function onBlockClick() {
        strCommand = "action=BLOCK_WORK";
```

```
            getDataFromServer();
            strCommand = "";
        }
        function setTime() {
            const now = new Date();
            // Get time parameters
            const second = now.getSeconds();
            const minute = now.getMinutes();
            const hour = now.getHours(); // In 0-23 format
            const dayOfWeek = now.getDay() === 0 ? 7 : now.getDay(); // Convert from 0-6 (in JS) to 1-7
            const dayOfMonth = now.getDate();
            const month = now.getMonth() + 1; // In JS, months start from 0
            const year = now.getFullYear() % 100; // Take the last two digits of the year
            // Form the request string
            strCommand =
`second=${second}&minute=${minute}&hour=${hour}&dayOfWeek=${dayOfWeek}&dayOfMonth=${dayOfMonth}&month=${mo
nth}&year=${year}`;
            getDataFromServer();
            strCommand = "";
        }


        // Start getting data when the page loads
        window.onload = function() {
            getDataFromServer();
        };
    </script>

    <style>
        body {
            font-family: Arial, sans-serif;
            text-align: center;
            margin: 0;
            padding: 0;
            background-color: #C2DFFF; /* Sea blue page background */
        }

        h1 {
            margin-top: 50px;
        }

        .variables {
            display: flex;
            justify-content: space-evenly;
            align-items: center;
            margin-top: 20px;
            flex-wrap: wrap;
        }

        .variable {
            font-size: 18px;
            margin: 0 20px;
        }

        .boxes-container {
            display: flex;
            justify-content: space-evenly;
            margin-top: 40px;
            flex-wrap: wrap;
        }

        .box {
            border: 2px solid black;
            padding: 20px;
            width: 30%;
            box-sizing: border-box;
            margin: 10px;
        }

        .box h2 {
            margin-top: 0;
        }
```

```
        .param {
            margin: 10px 0;
        }

        .param label {
            margin-right: 10px;
            font-size: 18px;
        }

        .box button {
            margin-top: 10px;
            padding: 10px;
            width: 100%;
            font-size: 18px;
            font-weight: bold;
        }

        .input-field {
            padding: 5px;
            width: 90%;
            font-size: 18px;
            font-weight: bold;
            text-align: center; /* Center values in fields */
            background-color: #FFFFFF; /* Initial background - white */
        }

        .box:nth-child(1) {
            background-color: #3BB9FF; /* Deep sky blue background for PUMP */
        }

        .box:nth-child(2) {
            background-color: #FFDB58; /* Mustard background for TIMERS */
        }

        .box:nth-child(3) {
            background-color: #FEFCFF; /* Milky white background for FORCED */
        }

        /* Media query for vertical format */
        @media (max-width: 768px) {
            .boxes-container {
                flex-direction: column; /* Vertical layout for boxes */
                align-items: center;
            }

            .box {
                width: 80%; /* Reduce box width on mobile devices */
            }
        }

        #time-to-off {
            margin-left: 10px;
            font-size: 18px;
            font-weight: bold;
        }

        .set-time-btn {
            padding: 8px 16px;
            font-size: 16px;
            margin-left: 10px;
        }
        .right-par {
            text-align: right;
            font-family: verdana;
            font-size: 15px;
            margin: 10px;
            margin-top: 50px;
        }
    </style>

</head>

<body>
```

```
  <h1>Water pumping and storage station</h1>

  <div class="variables">
    <div class="variable" id="temperature">Temperature: ...</div>
    <div class="variable" id="humidity">Humidity: ...</div>
    <div class="variable" id="data">Data: ...</div>
    <div class="variable" id="time">Time: ...</div>
    <button class="set-time-btn" onclick="setTime()">Set Time</button>
  </div>

  <div class="boxes-container">
    <!-- PUMP Box -->
    <div class="box">
      <h2>PUMP</h2>
      <div class="param">
        <label for="voltage">Voltage is:</label>
        <input type="text" id="voltage" class="input-field">
      </div>
      <div class="param">
        <label for="rms">RMS current:</label>
        <input type="text" id="rms" class="input-field">
      </div>
      <div class="param">
        <label for="pressure">Pressure:</label>
        <input type="text" id="pressure" class="input-field">
      </div>
    </div>

    <!-- TIMERS Box -->
    <div class="box">
      <h2>TIMERS</h2>
      <div class="param">
        <label for="last-time">Last on time:</label>
        <input type="text" id="last-time" class="input-field">
      </div>
      <div class="param">
        <label for="time-per-day">Time per Day:</label>
        <input type="text" id="time-per-day" class="input-field">
      </div>
      <div class="param">
        <label for="starts-per-day">Number starts per day:</label>
        <input type="text" id="starts-per-day" class="input-field">
      </div>
    </div>

    <!-- FORCED Box -->
    <div class="box">
      <h2>FORCED</h2>
      <button id="start-btn" onclick="onStartClick()">START <span id="time-to-off"></span></button>
      <button id="stop-btn" onclick="onStopClick()">STOP</button>
      <button id="block-btn" onclick="onBlockClick()">BLOCK WORK</button>
    </div>
  </div>

  <footer>
                <p class="right-par">© Ochilnik - WPS 2025</p>
        </footer>

</body>
</html>
```

Listing 17 - Web Interface Module

6 Operational Description

The system's primary function is to monitor the state of the pump station (WPS), track its operating time, display parameters, and signal alarm conditions.

The operating time is counted based on the presence of supply voltage to the WPS. After the WPS is turned off, the counters retain their last values. The following counters are implemented:

The last run time counter tracks the pump's operating time for a single 'ON' cycle. It resets each time a new count begins.

The daily runtime counter tracks the total daily operating time of the pump. It resets after the first 'ON' cycle of the next day.

The daily start counter tracks the number of 'ON' cycles for the current day. It resets after the first 'ON' cycle of the next day.

The countdown timer shows the time remaining until the pump shuts off after being started by the START command. After the time delay expires (1 min) or after a STOP command, it resets to its initial value of 60 seconds.

The current values of the counters are displayed on screen 1 of the MCC controller's display and on the web page.

Figure 22 – MCC Control and Indication Assignment

The other monitored parameters include the root mean square (RMS) current consumption of the pump station motor, and the air temperature and humidity in the pump station room. These parameters are displayed on screen 2 of the MCC controller's display and on the web page.

Figure 23 – MCC Display Screen 2 Indication Assignment

Screen 3 of the MCC controller display cycles through error messages.

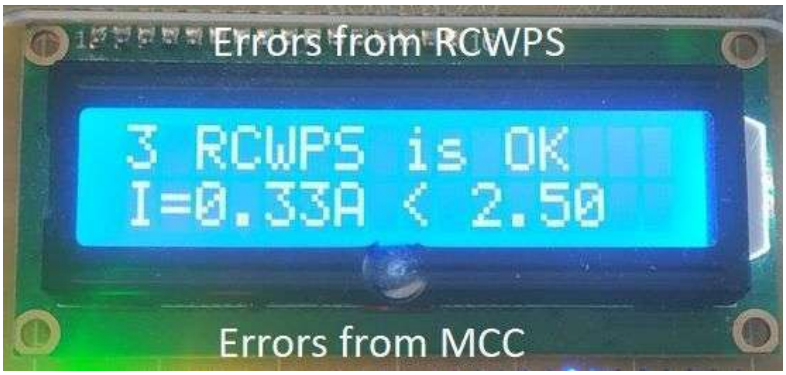


Figure 24 – MCC Display Screen 3 Indication Assignment

In the current implementation, the detection and signaling of the following faults are provided:

Table 5 – List of Faults Indicated by the WPS Control System

No	Fault Description	Display Message
1	Supply voltage is absent after starting the pump with the START command	RelayON noVolt
2	Supply voltage is present after enabling the start block with the BLOCK WORK command	BlockON VoltON
3	Single run time exceeds 10 min	Last ON > 10 min
4	Runtime for the current day exceeds 1 hour	Day ON > 60 min
5	Average motor current consumption exceeds 2.95A (motor overload, I_{nom}=2.8A)	I=X.XXA > 2,95A
6	Average motor current consumption is below 2.5A (this value indicates the pump is running dry)	I=X.XXA < 2,5A
7	Room temperature is above 30°C	T=XX.XC > 30C

In addition to the display, there are three LED indicators on the MCC controller panel.

Voltage – indicates that supply voltage is present (green).

Error – indicates a fault condition (yellow).

Block – indicates that the pump operation is blocked (red).

The RCWPS controller has an error status indicator, an AC 220V voltage indicator, and "Start ON" and "Block ON" indicators. The controller's buttons are equipped with their own built-in indicators.

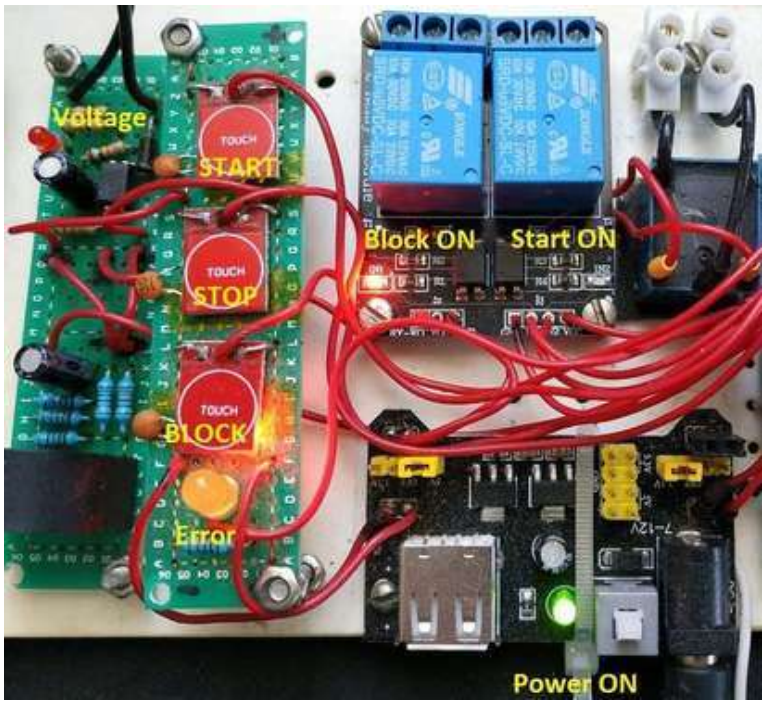


Figure 25 – RCWPS Control and Indication Assignment

Forced control of the pump station is provided for manually starting the pump and for forcibly blocking its operation, regardless of the state of the pressure switch that controls the pump.

Controls are provided on both controllers and are duplicated in the web interface. The button assignments are as follows:

START – starts the pump for a limited time (1 min). This limitation is implemented to prevent fault conditions during prolonged pump operation, as this command bypasses the pressure switch.

STOP – stops a pump that was started by the previous command. It does not affect operation if the pump was started by the pressure switch.

BLOCK WORK – blocks the pump from starting. If the pump is running, this command stops it. The button operates in a toggle mode; a second press is required to disable the block.

MENU – this button exists only on the central controller. It is used to switch between screens on the display. It also activates the display backlight for a limited time (2 min). For easier access, it is implemented as a touch button, with the bottom-right mounting screw serving as the touch electrode.

The web interface replicates the controls and indicators mentioned above. The indication and control elements are divided into groups: Date and Time, Pump Parameters, Counters, and Control.

Via the web interface, the "Set Time" button allows for synchronizing the control system's internal clock with the client's time.

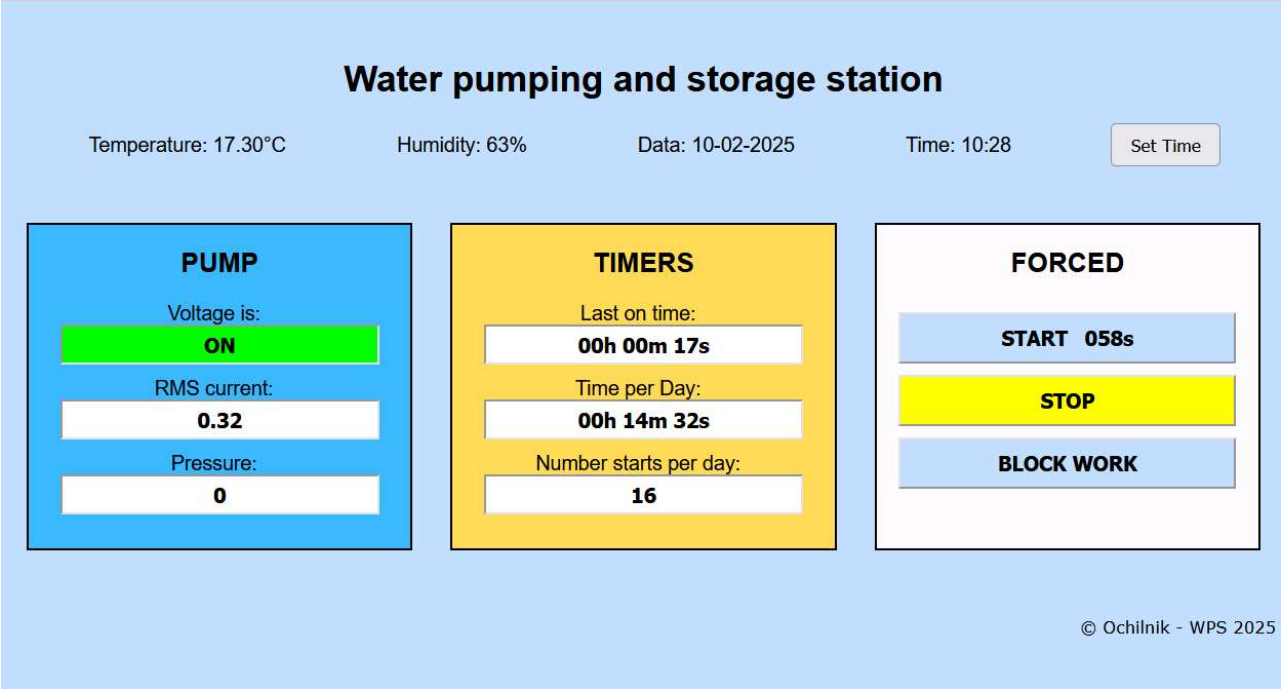


Figure 26 – Web Interface of the WPS Control System

State indication is achieved by highlighting specific fields.

Table 6 – Display of System Status Indication on the Web Interface

No.	Indication	Meaning
1	Green color of the "Voltage is" field	Voltage is applied
2	Yellow color of the "RMS Current" field	Current deviation
3	Yellow color of the "Last on time" field	Time has exceeded 10 min
4	Yellow color of the "Time per Day" field	Time has exceeded 60 min
5	Green color of the START button	Forced start is active
6	Yellow color of the STOP button	Fault condition
7	Red color of the BLOCK button	Block is enabled

The combined operation of all system components on a test bench is demonstrated in the following video: https://github.com/Ochilnik/WPS/blob/master/WPS.mkv.

7 Future Upgrades

The described capabilities of the pump station control system are not perfect or final. It can be considered the first version, ready for operational testing.

Not all of the initial ideas have been implemented. Some ideas emerged during the implementation process, some during the writing of this report, and more will likely appear in the future. Among the main prospects for modernization, the following should be noted:

Equipping the pump station with a pressure sensor with an analog output. Information about the pressure will allow for:

- unambiguously reacting to pressure changes beyond the operating limits;
- accounting for shifts in operating limits depending on the ambient air temperature;
- predicting a drop in air pressure in the accumulator in advance;
- fully implementing the functions of an intelligent pressure switch.

It is possible to measure water pressure with a hydraulic sensor or to measure air pressure in the accumulator with a pneumatic pressure sensor.

For this purpose, an analog input on the RCWPS controller, space in the communication protocol, and a field for indication and visualization have been reserved.

Equipping the pump station with a discrete leak sensor. This will allow for a quick and unambiguous response to emergency situations by shutting down the pump.

Optimizing network communication with the client. This will free up the not-so-powerful network interface of the MCC controller for additional tasks.

Creating and maintaining a database of pump station parameters. This will allow for storing operational data over a long period, performing data sampling, building graphs, comparative dependencies, and more.